

AUTOMATIZACIÓN E IA CON LOVABLE Y N8N

TEMA 3. Fundamentos de n8n

icono TRAINING CONSULTING

Contenidos Tema 1

/ 1. Introducción al ecosistema n8n en 2026

/ 2. Puesta a punto con n8n cloud

/ 3. Diseño de workflows básicos: triggers, nodos y conexiones

/ 4. Sistema Draft vs. Production

/ 5. Conexión n8n con APIs

/ 6. PRÁCTICA: configuración Bot de Telegram con respuestas automáticas

/ 7. Manejo de datos y expresiones en n8n

¿QUÉ SON LAS HERRAMIENTAS DE AUTOMATIZACIÓN?

Se trata de herramientas software que nos permiten conectar apps y diferentes sistemas entre sí, y ejecutar tareas o flujos de trabajo repetitivos de forma automática.

¿POR QUÉ SU RELEVANCIA EN LA ACTUALIDAD?

- Permiten ahorrar tiempo y reducen esfuerzo humano para realizar tareas repetitivas.
- Ayudan a escalar procesos de negocio: notificaciones, sincronización de fuentes de datos, informes, *pipelines* sencillos con AI/ML, etc.

En definitiva, el ROI de un negocio puede verse mejorado, pues las automatizaciones aligeran la carga de roles como los desarrolladores y analistas, y mejoran la consistencia de procesos

n8n

QUÉ PERMITEN LAS HERRAMIENTAS DE AUTOMATIZACIÓN

- Conectar apps entre sí (APIs, webhooks), transformar datos, planificar disparadores (triggers), lógica condicional (SI sucede X, hacer Y), gestión de errores.
- Ampliables mediante código o modelos de IA para flujos más avanzados.

HERRAMIENTAS MÁS COMUNES

- **Zapier:** interfaz de usuario amigable, automatizaciones app-to-app orientadas a negocio
- **Make:** constructor visual de flujos potente para ramificaciones complejas de procesos
- **Microsoft Power Automate:** integración profunda con Microsoft 365 / Enterprise
- **n8n**

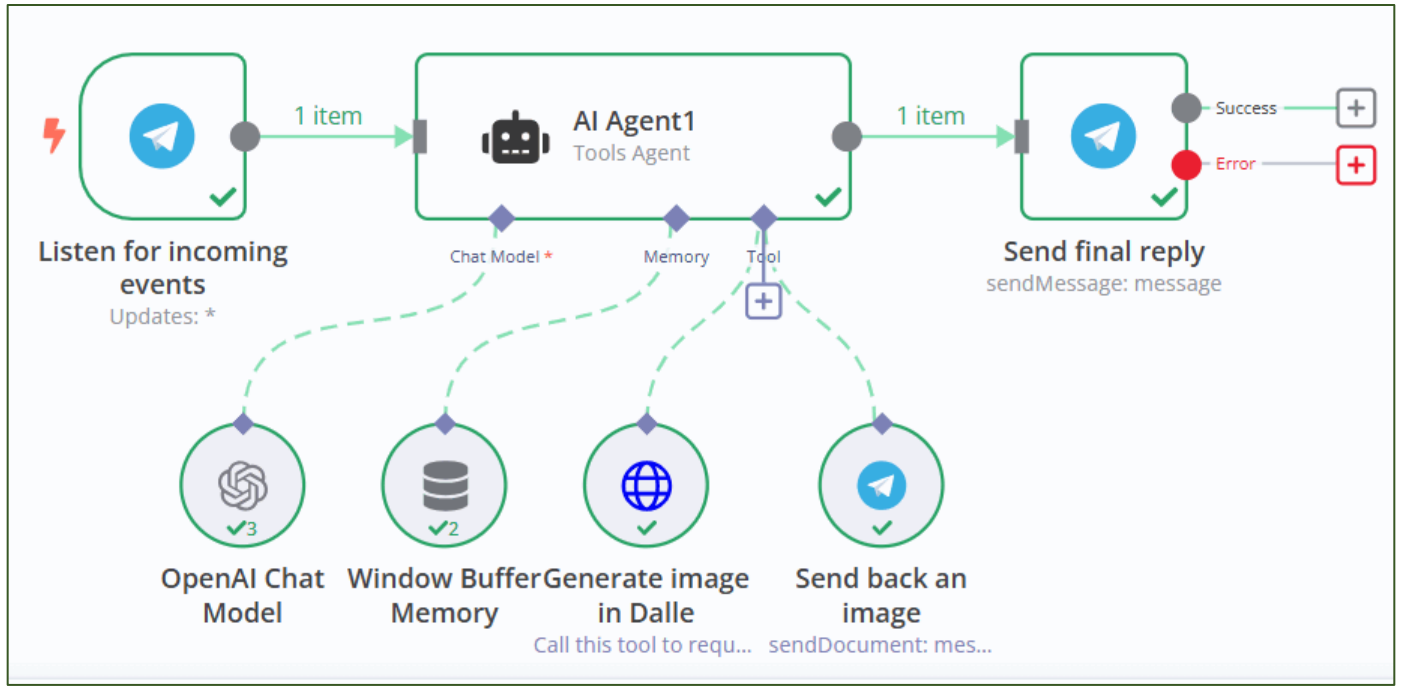
zapier

make

Power Automate

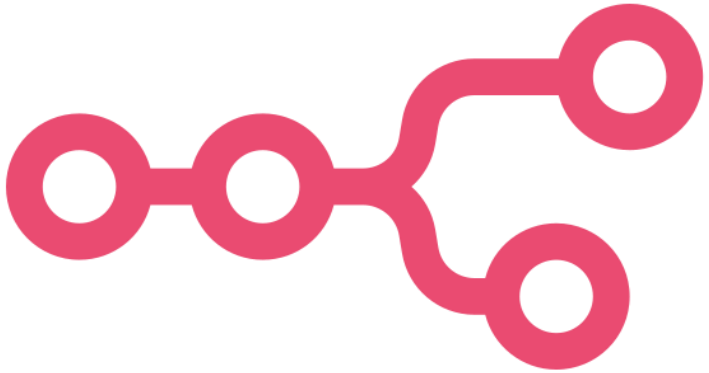
PRESENTANDO N8N

- Herramienta para automatizaciones que ha ganado gran popularidad en los últimos años.
- Plataforma extensible de automatización de flujos de trabajo, con foco en dar soporte a equipos técnicos.
- Editor visual basado en nodos.
- Puede usarse de forma auto-alojada (self-hosted) o bien como un servicio gestionado en la nube.

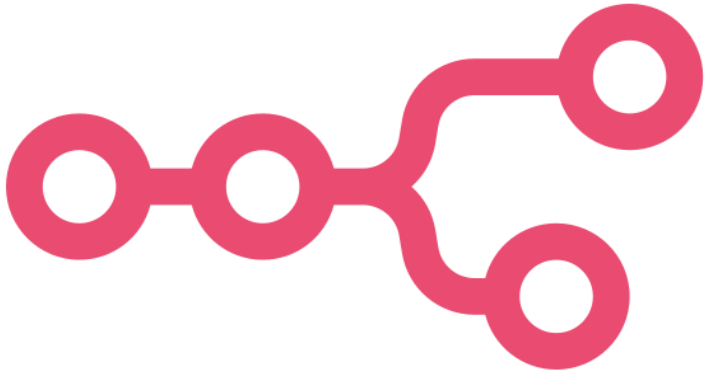


Casos de uso habituales: pipelines de análisis de datos, automatización de procesos internos, pipelines con IA y agentes, integraciones customizadas, etc.

En 2026, n8n ha dejado de ser un mero "*Zapier para desarrolladores*" para pasar a ser el cerebro orquestador preferido para crear Agentes de IA y conectarlos con datos privados.



En el último tramo de 2025, n8n alcanzó una valoración histórica de $2.300 millones de dólares, consolidándose como un unicornio tecnológico gracias a su altísima flexibilidad.



¿QUÉ DISTINGUE A N8N DE OTRAS HERRAMIENTAS?

- **De código abierto y con la posibilidad de alojarla en máquina propia**: control total sobre los datos, sin límites de ejecuciones cuando se aloja en máquina propia, foco en privacidad y altos volúmenes de usuarios.
- **Alta extensibilidad y foco en desarrolladores**: amplio ecosistema de nodos, habilidad de añadir integraciones personalizadas, control de versiones (Git), funcionalidades para equipos.
- **IA & Soporte para Flujos Avanzados**: posicionada para flujos modernos con IA (conectar modelos de IA, orquestar tareas llevadas a cabo por agentes autónomos).
- **Dificultades:** mayor carga de configuración técnica frente a competidores SaaS

  → Versión *cloud* lo mitiga, pero se pierde parte del control sobre los datos y la privacidad.

RADIOGRAFÍA DE LA ADOPCIÓN MASIVA DE N8N

- +230.000 usuarios globales
- +3000 despliegues a nivel corporativo y de empresa
- **La Comunidad**: Uno de los proyectos con mayor tracción en la historia de *GitHub*, con cientos de sub-comunidades creadas por los usuarios para conectar todo tipo de apps de terceros.
- **Enfoque visual:** n8n utiliza un "lienzo infinito" donde la complejidad visual es, al mismo tiempo, flexibilidad casi absoluta.

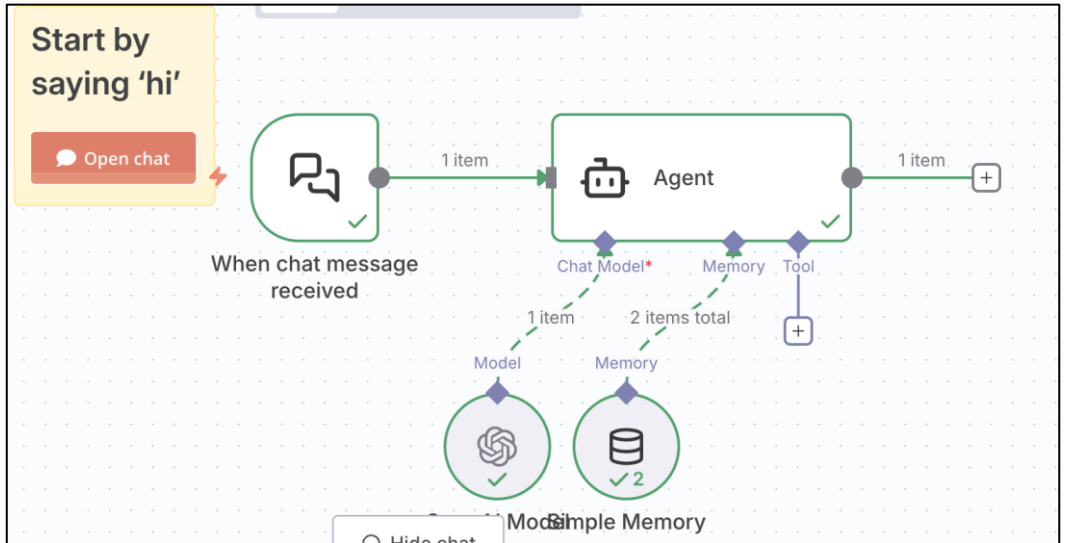


¿QUÉ ES UN AGENTE EN N8N?

En n8n, un **agente** es un asistente software autónomo, con la capacidad de:

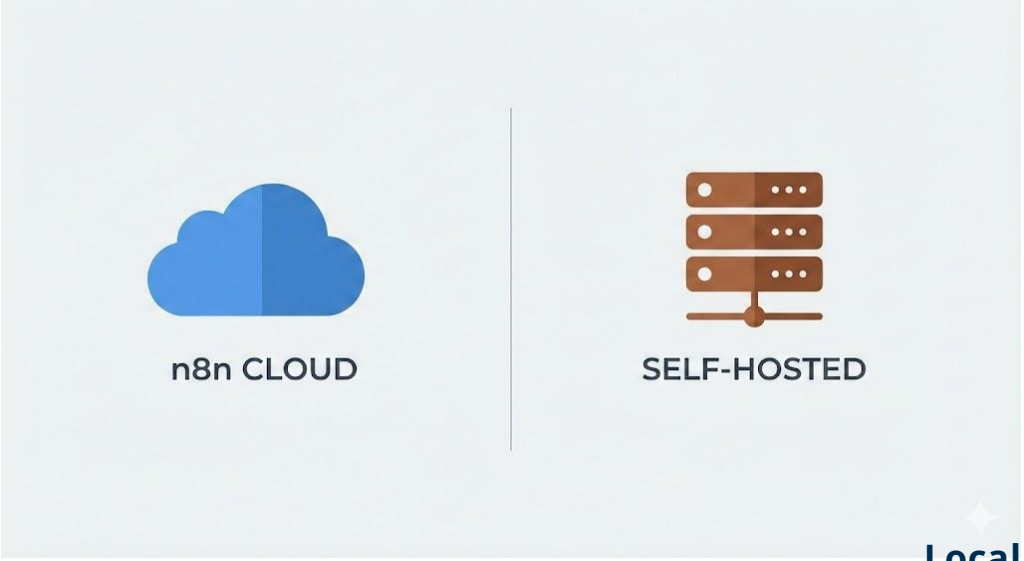
- Recibir y procesar instrucciones en lenguaje natural (dotado de modelos de NLP).
- Enviar mensajes/peticiones a herramientas externas mediante APIs.
- Buscar información en fuentes de datos.
- Ejecutar tareas dentro de un flujo definido por nosotros.

IMPORTANTE: un agente no puede hacer nada de lo descrito por sí solo. Deberá ser parte integral de un flujo, estando conectando a nodos (principalmente a LLMs*) y herramientas que le brindarán la capacidad de actuar.



***LLMs (Large Language Models)**
Aplicaciones conversacionales modernas
Como Gemini, ChatGPT, etc.

N8N AUTO-GESTIONADO (self-hosted) VS. EN LA NUBE (cloud)



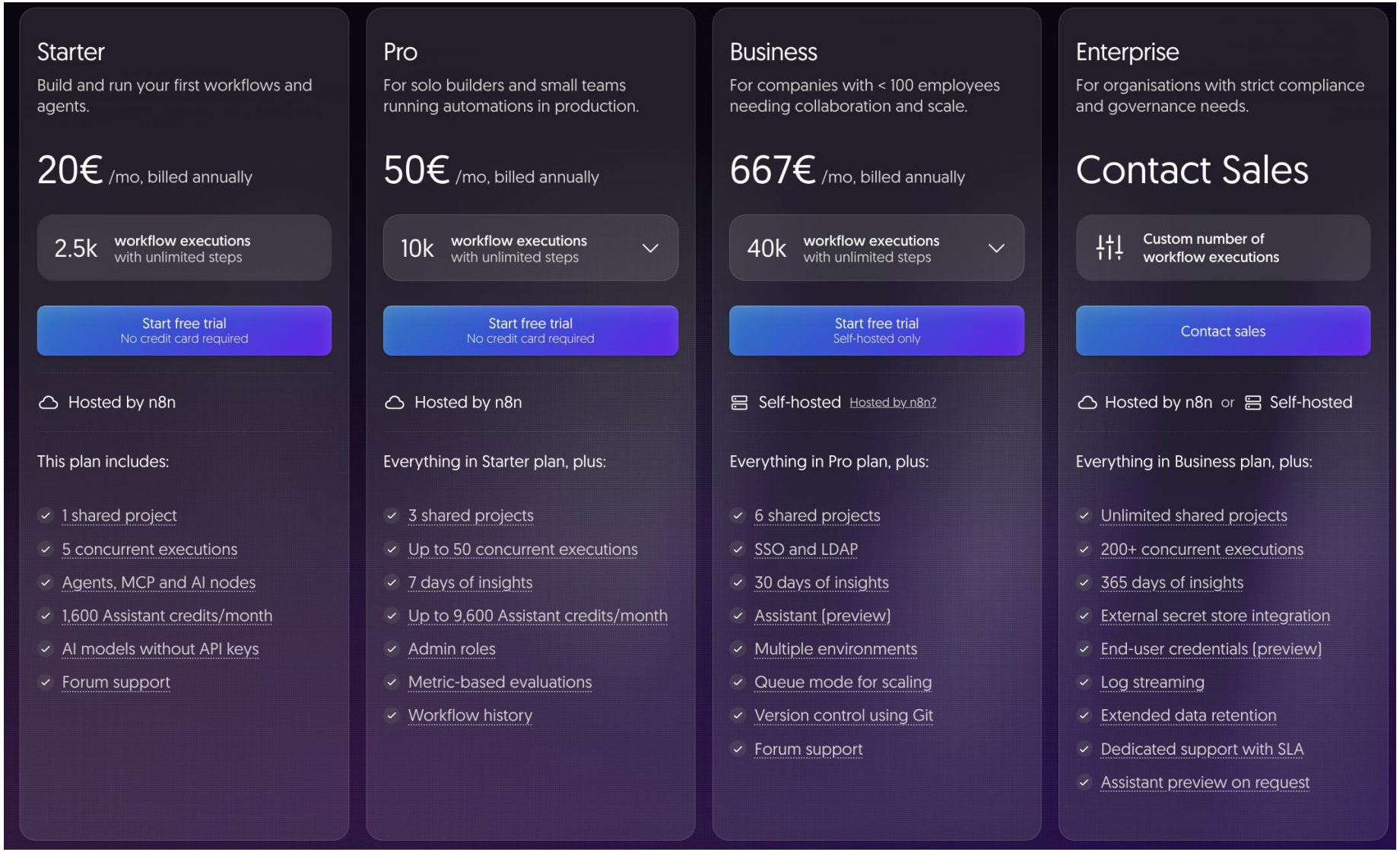
En la nube gestionada por la propia n8n

- Sin pre-configuración técnica.
- Infraestructura escalable
- Actualizaciones automáticas.
- Listo para empezar a construir flujos desde el minuto cero.

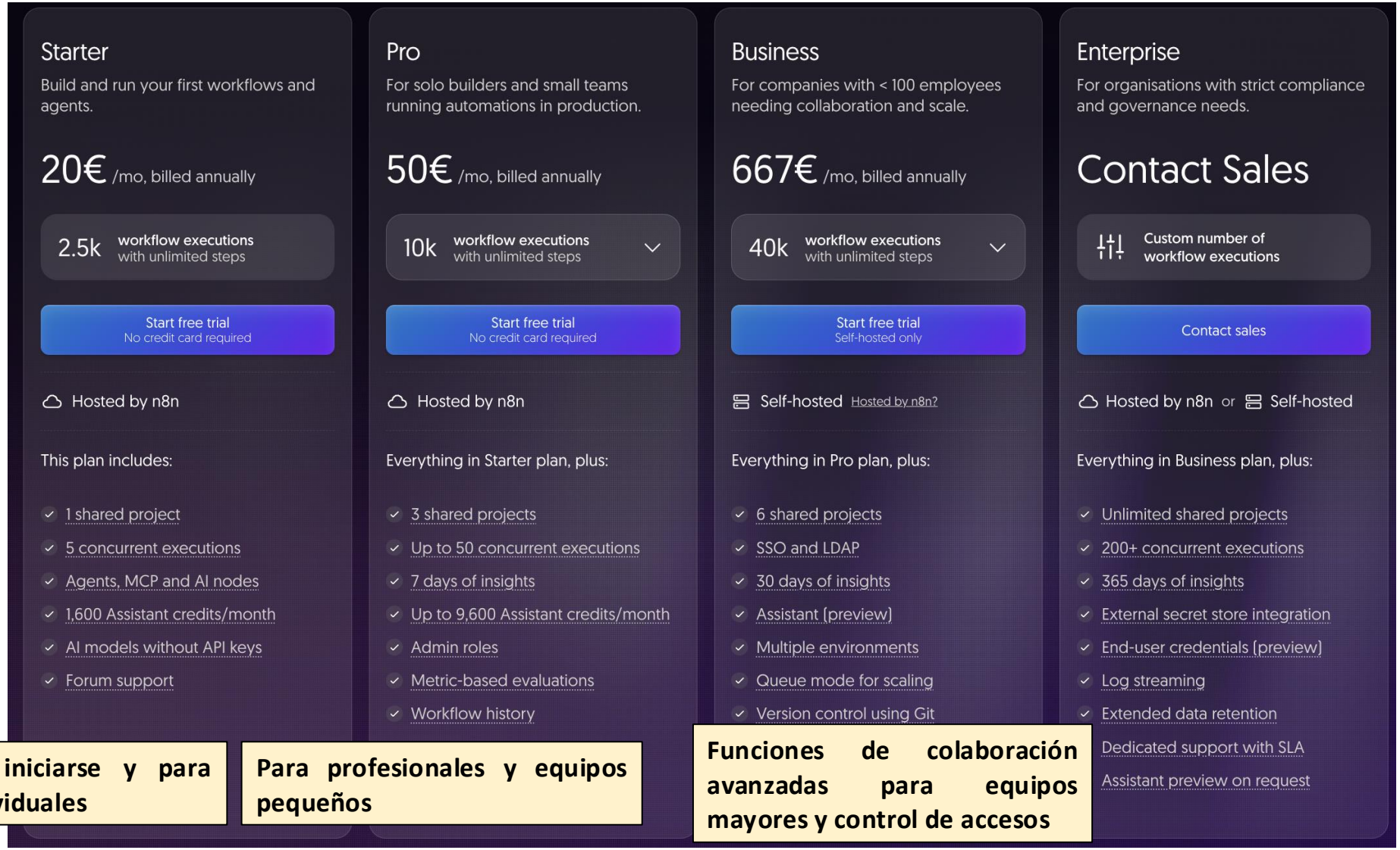
Local o en servidor propio, p.ej. Máquina en AWS

- Gratis
- Control total de datos y privacidad extrema.
- Requiere mantenimiento técnico y gestión de infraestructura (p.ej. Docker) y actualizaciones manuales.

EL MODELO "FREEMIUM" DE N8N CLOUD: https://n8n.io/pricing/



EL MODELO "FREEMIUM" DE N8N CLOUD: https://n8n.io/pricing/

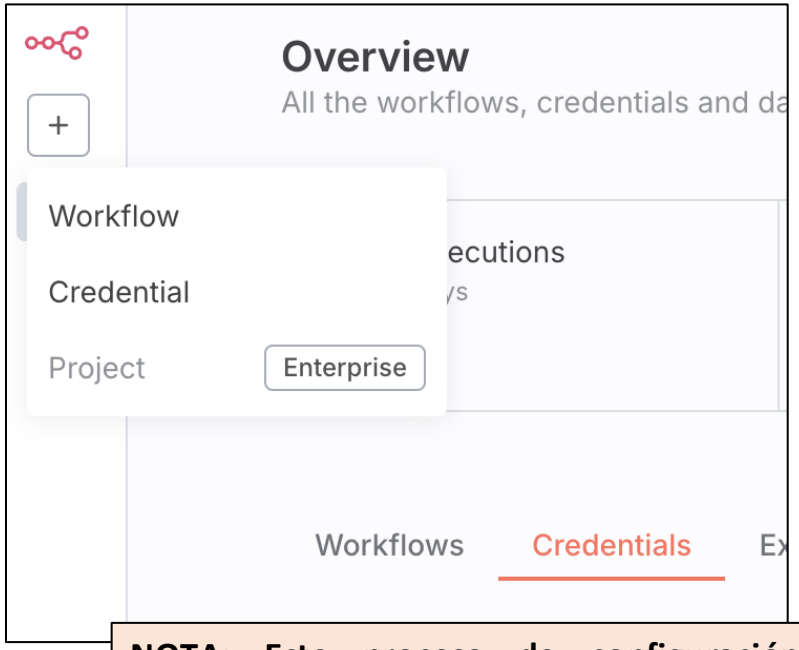


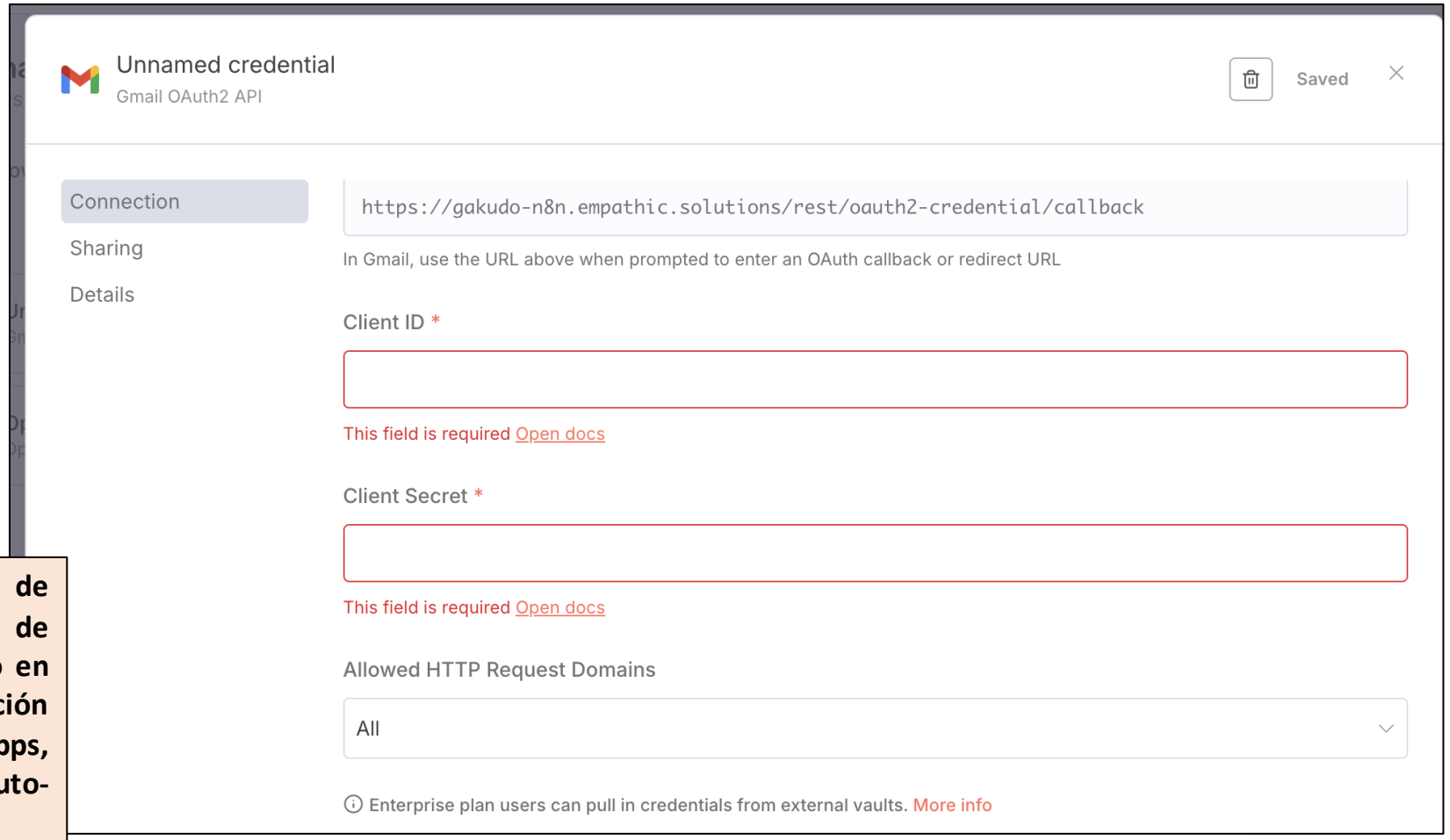
IDEAL para iniciarse y para usuarios individuales

Para profesionales y equipos pequeños

Funciones de colaboración avanzadas para equipos mayores y control de accesos

Ya dentro de n8n, uno de los aspectos que deberemos abordar es el de las credenciales → se definen para autorizar la autenticación, por parte de n8n, de conectar n8n con diferentes Apps como Gmail y muchas otras



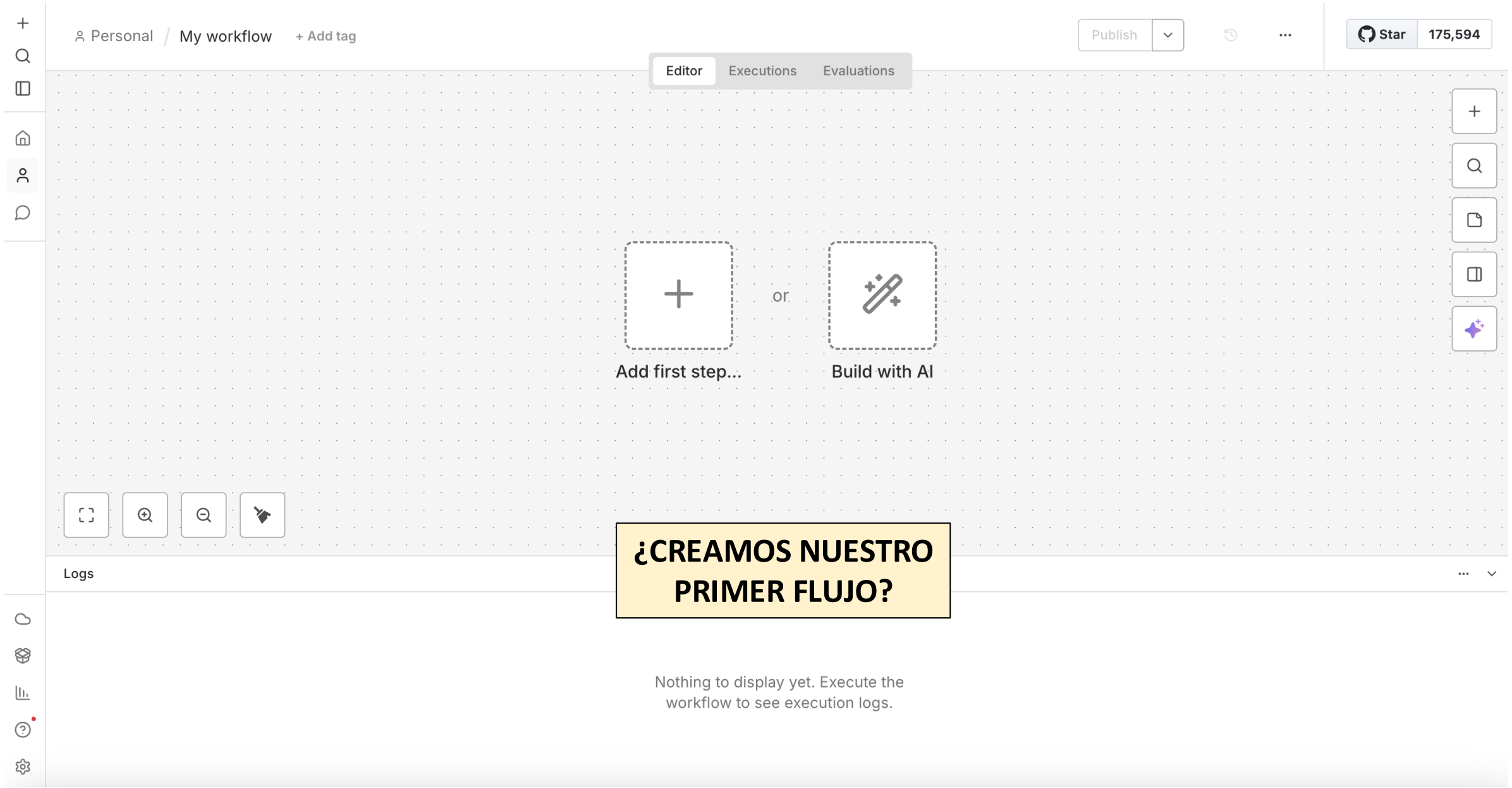


NOTA: Este proceso de configuración de credenciales para varias Apps, p.ej. Las de Google, ha sido notablemente simplificado en la versión actual de n8n cloud. La configuración manual aquí mostrada es, para algunas apps, requerida únicamente en la versión auto-gestionada (self-hosted) de n8n

LOS TRES PILARES DE LA AUTOMATIZACIÓN EN N8N

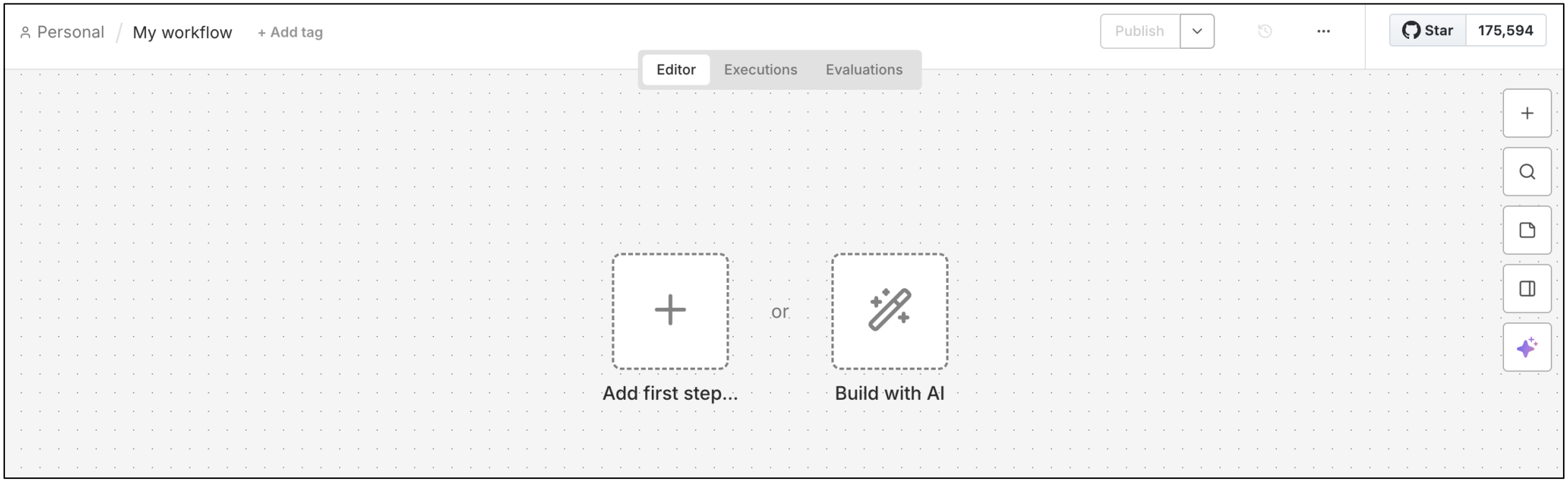
- **Triggers** (*iniciadores*): desencadenan o inician la ejecución de un flujo consistente en nodos que realizan distintas acciones. **¿Cuándo?** De forma periódica, manual, o a raíz de un evento externo, etc.
- **Nodos:** son las piezas del rompecabezas que constituye un flujo en n8n, y en cada uno se encapsula *qué* se hace. Ejemplos: filtrar datos, transformar texto, realizar cálculos matemáticos, comunicarse con otras plataformas, etc.
- **Conexiones:** el "flujo sanguíneo" de un workflow en n8n. Unen los nodos entre sí, definiendo y estructurando la secuencia de acciones a realizar.




Personal / My workflow + Add tag
Publish
Star 175,594
Editor Executions Evaluations
Add first step...
or
Build with AI
Logs
¿CREAMOS NUESTRO PRIMER FLUJO?
Nothing to display yet. Execute the workflow to see execution logs.

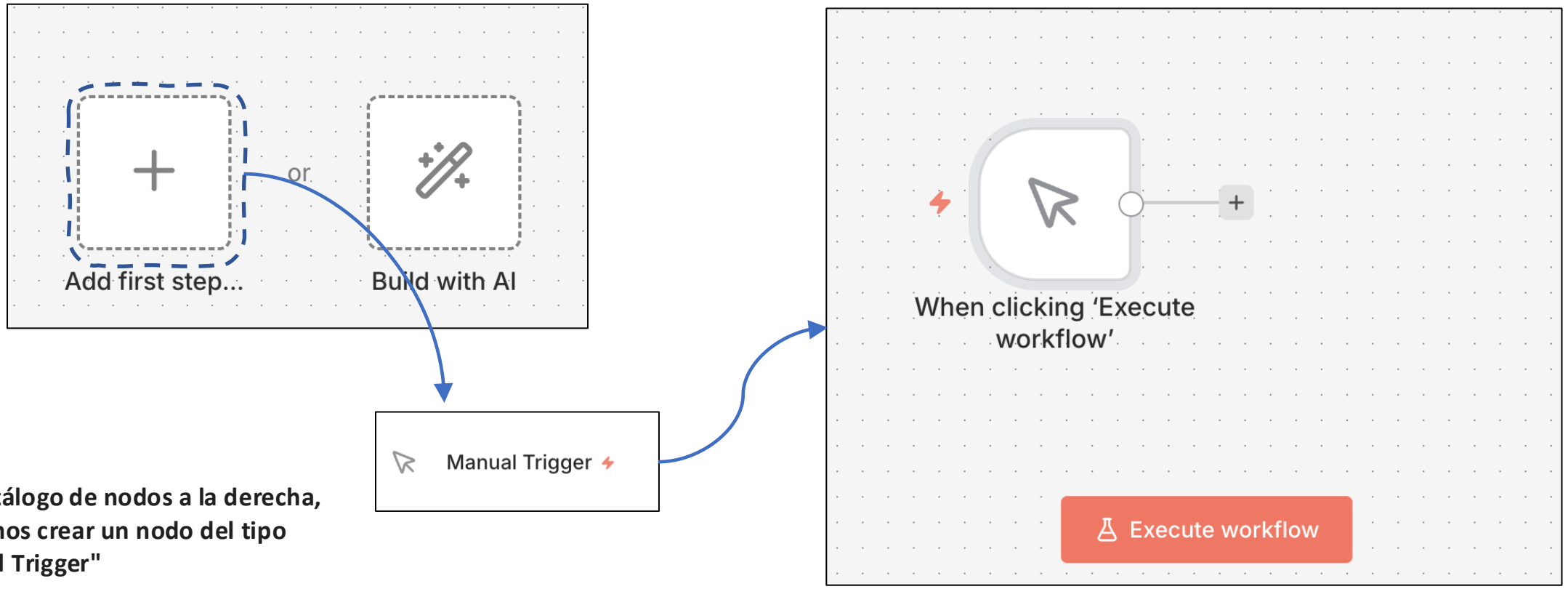
CONSTRUYENDO EL FLUJO MÁS SIMPLE DEL MUNDO

- Todo comienza con la creación de un nuevo *workflow* y un lienzo vacío, en el que añadiremos el primer nodo (normalmente el *trigger* o desencadenante) haciendo click en "**Add first step**":



CONSTRUYENDO EL FLUJO MÁS SIMPLE DEL MUNDO

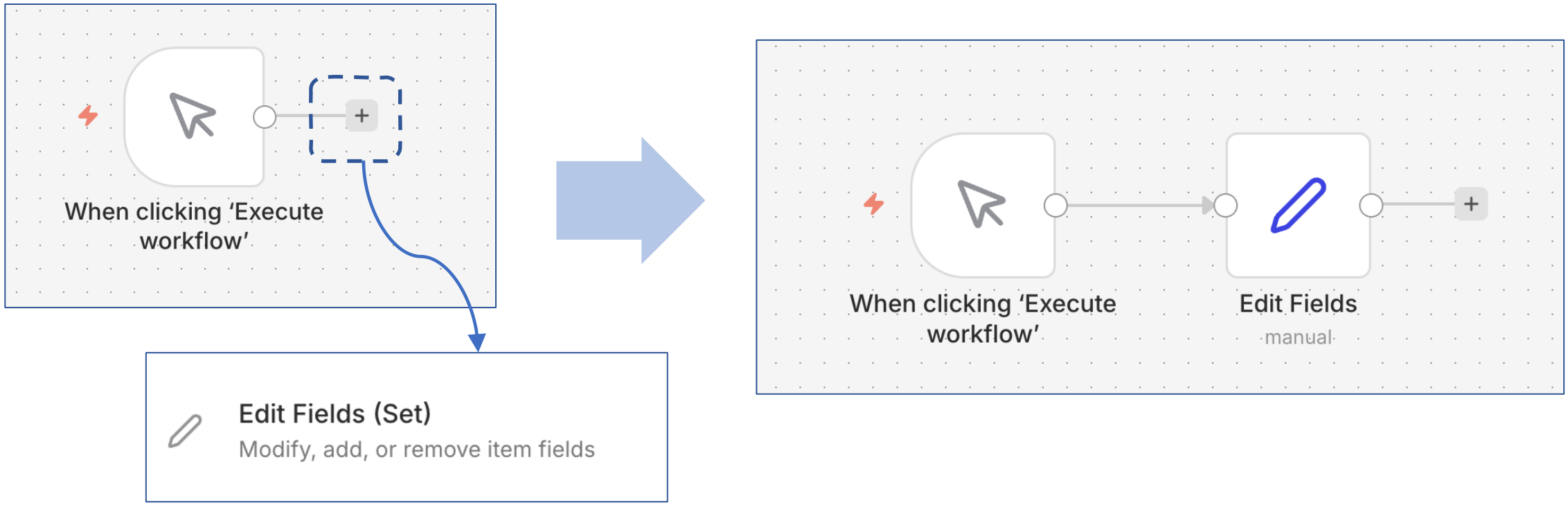
- Elegiremos un *trigger* manual: el más sencillo de todos. Simplemente, inicia la ejecución del flujo cuando hagamos click en el botón inferior "Execute workflow" en nuestro lienzo.



En el catálogo de nodos a la derecha, elegiremos crear un nodo del tipo "Manual Trigger"

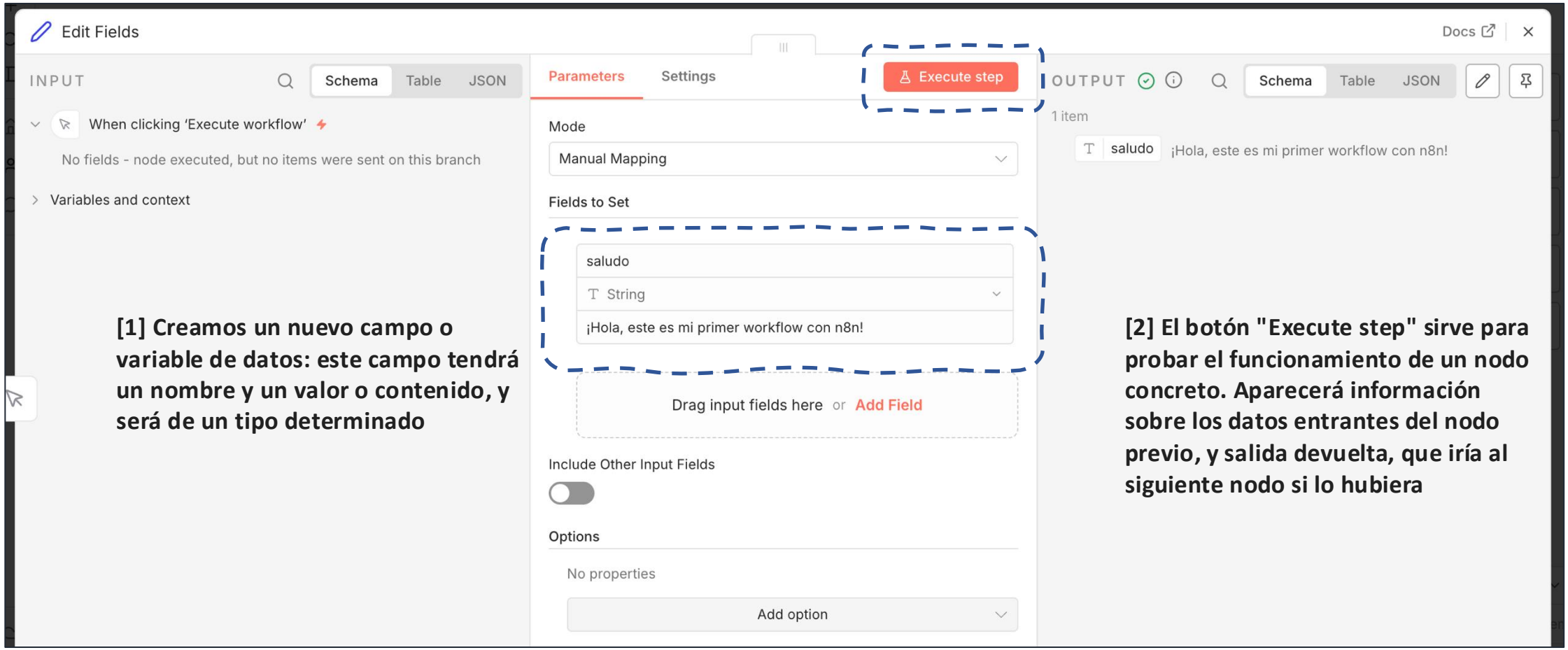
CONSTRUYENDO EL FLUJO MÁS SIMPLE DEL MUNDO

- Añadimos un nuevo nodo, también uno de los más sencillos de n8n → ***Edit Fields***
- Es un tipo de nodos que permite recoger, transmitir, y seleccionar datos entre nodos, pero también puede usarse para crear nuevas variables de datos.



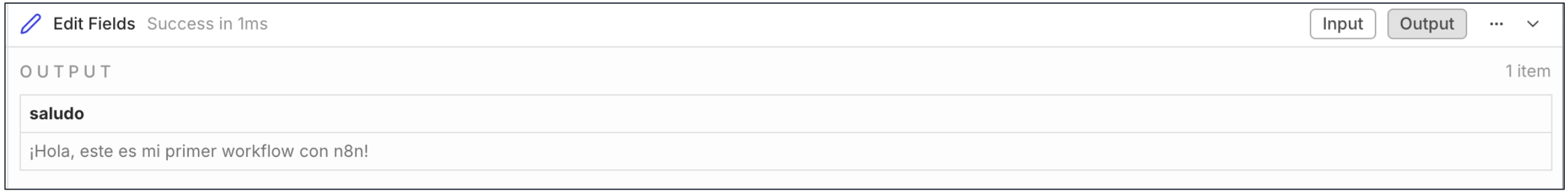
CONSTRUYENDO EL FLUJO MÁS SIMPLE DEL MUNDO

- Vista de configuración de un nodo: se abre automáticamente al crear un nodo, pero se puede reabrir en cualquier momento haciendo doble click sobre un nodo.



CONSTRUYENDO EL FLUJO MÁS SIMPLE DEL MUNDO (o tal vez, del universo...)

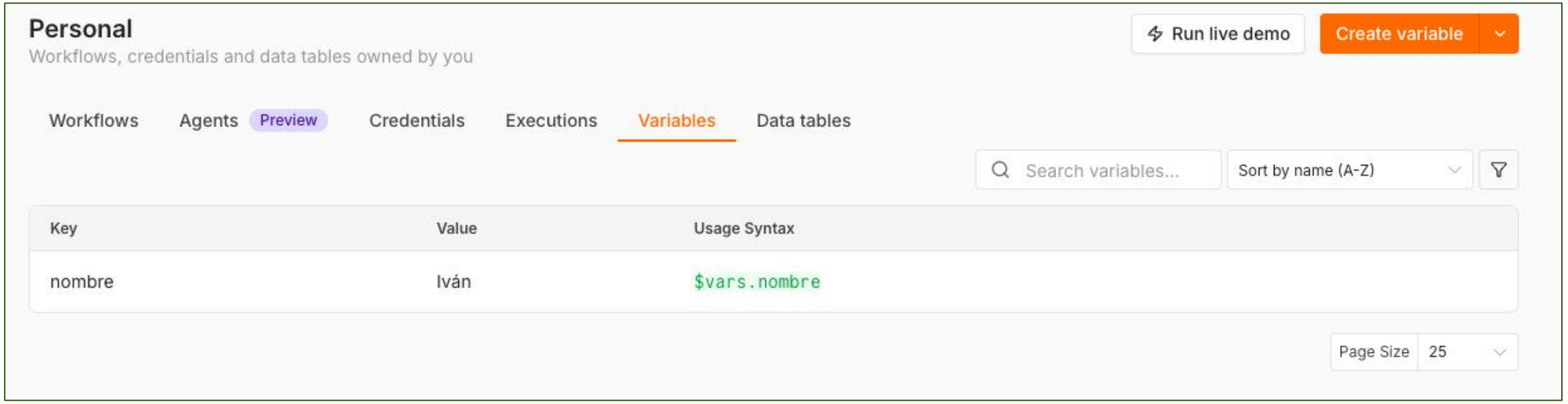
- Ejecutemos el flujo al completo desde el lienzo.
- El resultado final es el 'OUTPUT' del último nodo:



VARIABLES EN N8N (Novedad en versión 3.0 de n8n Cloud, desde Octubre 2026)

Centralización y reutilización de datos sin fricción:

- Contienen valores (cadenas de texto de hasta 1000 caracteres) accesibles y reutilizables desde cualquier flujo en nuestro espacio de trabajo de n8n.
- Son **inmutables:** la propia variable global definida en tu n8n no es modificable, pero desde un flujo puedes extraer su valor y guardarlo en otra variable local al flujo, que sí será manipulable.



EL MODO "CONSTRUCCIÓN" (Draft)

Es el entorno seguro para construir flujos. Aquí nos moveremos a nivel práctico prácticamente durante todo este curso de introducción a n8n.

- Cuando creamos un workflow nuevo, por defecto está *inactivo* (Draft).
- **Ejecuciones de prueba**: En este estado, podemos probar nodo por nodo o el flujo completo, empleando datos de prueba.
- **A prueba de fallos**: se trata de una zona de experimentación segura en la mayoría de casos.



DESPLIEGUE A PRODUCCIÓN (Active / Publish)

Dejar que la maquinaria trabaje por si sola cuando ya tenemos un flujo funcionando perfectamente.

- Una vez testeado y comprobado que nuestro flujo cumple su cometido, activamos la opción "**Publish**", en la parte superior del lienzo, para lanzarlo a producción.
- Desde este momento, el flujo está activo y "escuchando" de forma permanente, y se ejecutará siempre que la condición del nodo Trigger se cumpla.
- No se puede hacer en todos los flujos. Con un *trigger* manual como el que hemos creado antes, desplegar el flujo no es posible.
- **VENTAJA EN N8N v2:** podemos seguir modificando el flujo en el lienzo (Draft) sin romper accidentalmente la versión ya está publicada y trabajando. Los cambios no se reflejan en la versión en producción hasta volver a darle a "*Publish*".

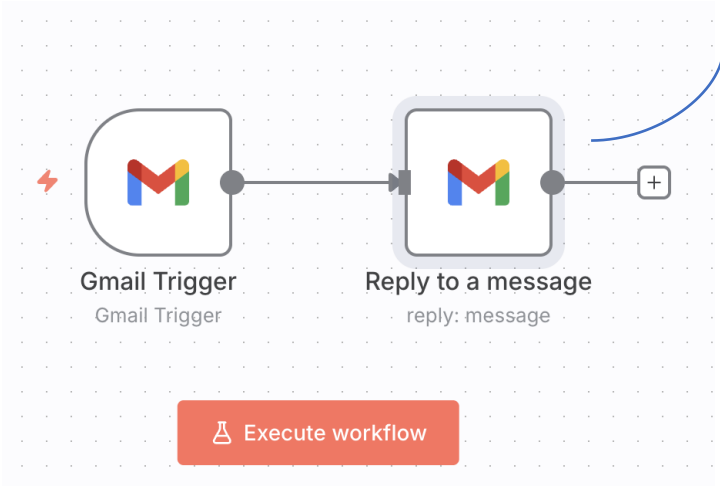


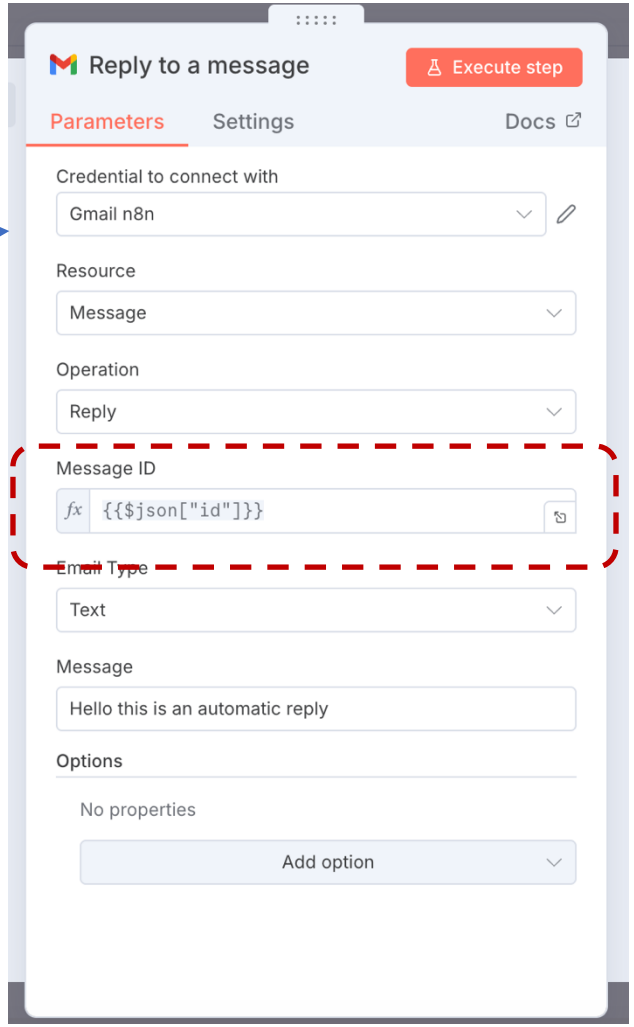
Expresiones entre doble llave, como por ejemplo:

`{{$json["id"]}}`

- Son evaluadas por n8n como si fueran código Javascript, usando datos que llegan desde el nodo anterior a aquel que estamos configurando.
- Por lo general, los nodos en n8n devuelven una estructura de datos similar a la siguiente:

```
{
  "id": "187b98323f92abc3",
  "threadId": "...",
  "subject": "...",
  ...
}
```





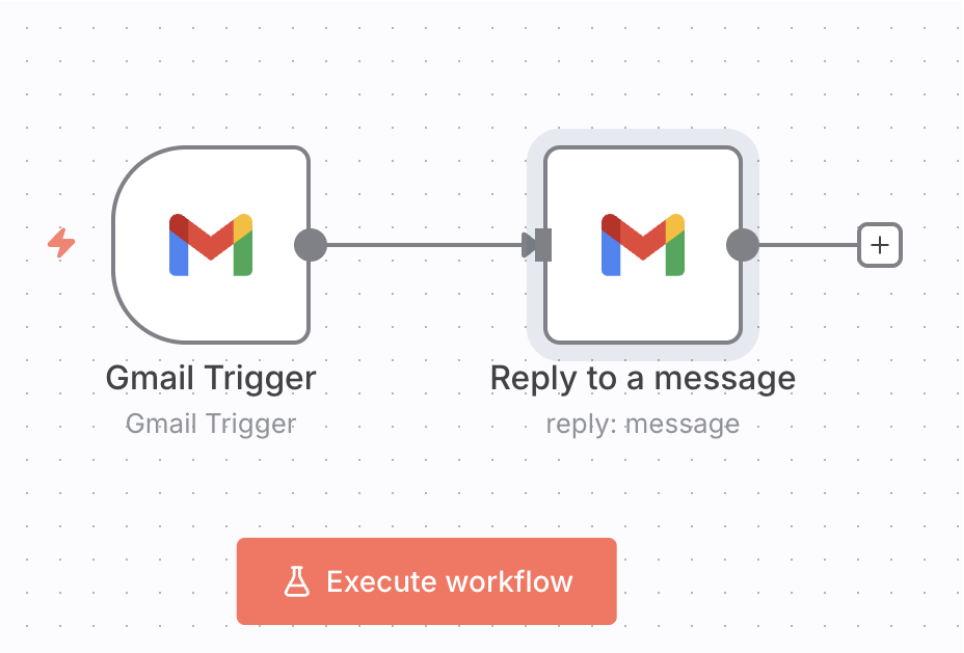
- Así, "`$json`" hace alusión al objeto JSON proveniente del nodo previo = datos de entrada del nodo actual.
- Usado en el campo 'Message ID' de un nodo como el de este flujo, selecciona los nuevos emails entrantes de la cuenta de Gmail vinculada

Otros ejemplos de expresiones similares:

- `{{$json["subject"]}}` → Obtiene el asunto del email entrante
- `{{$json["payload"]["headers"][0]["value"]}}` → acceso a objetos anidados
- `$node["Node Name"].json ->` los datos JSON de cualquier otro nodo
- Combinar texto fijo con variables: `Hello, the message ID is: {{$json["id"]}}`
- Generación de timestamps: `{{ new Date().toISOString() }}`

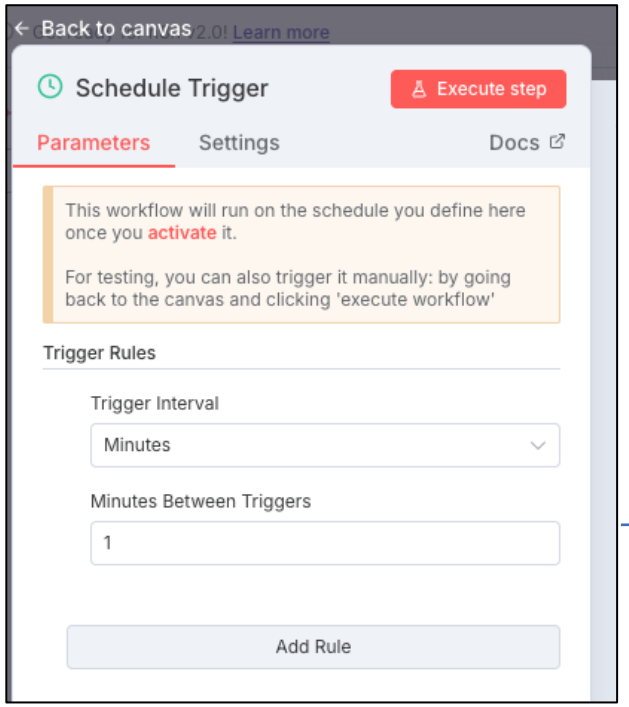
Uso de expresiones en nombres de archivos creados, URLs, emails, etc.:

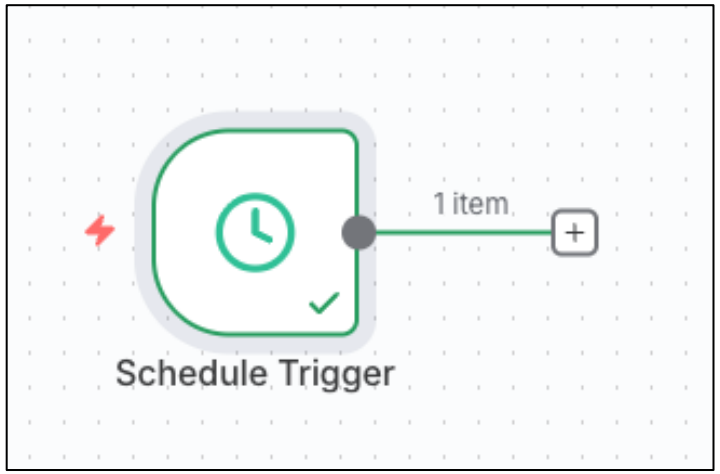
- `Report-{{$now}}.txt`



¿Y SI AHORA QUEREMOS CONECTAR n8n CON OTRA APP DE GOOGLE, P.EJ. Google Sheets?

- Ahora podemos, en un nuevo flujo de trabajo, crear nodos de Google Sheets.
- Por ejemplo, nuestro primer nodo (Trigger) puede ser del tipo "Schedule Trigger" (antes llamado 'cron')





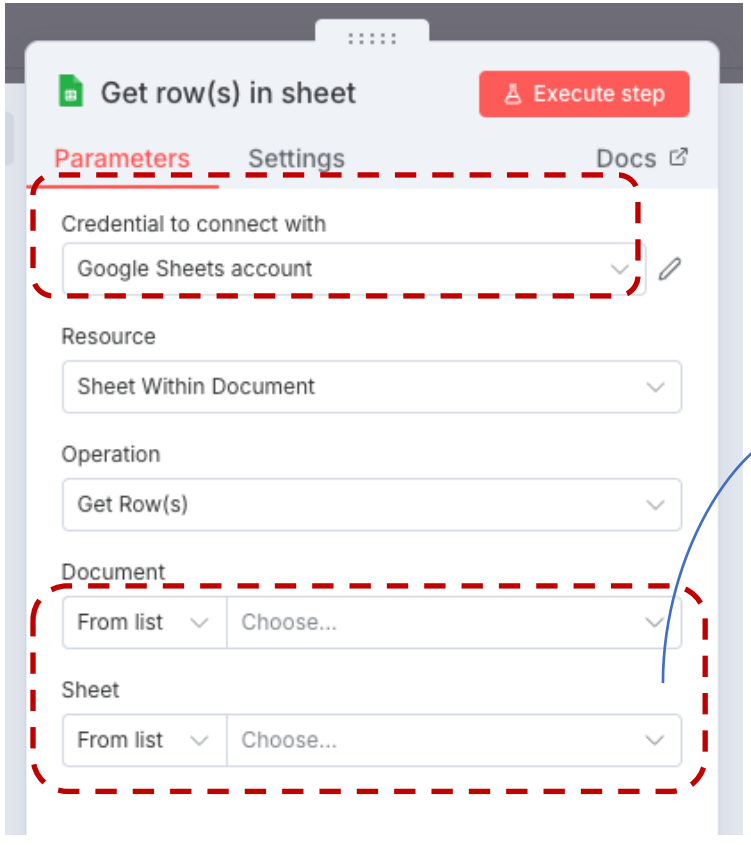
Este nodo se usa para que se ejecute el resto del flujo al que vaya conectado en instantes/intervalos de tiempo fijados, p.ej. una vez por minuto

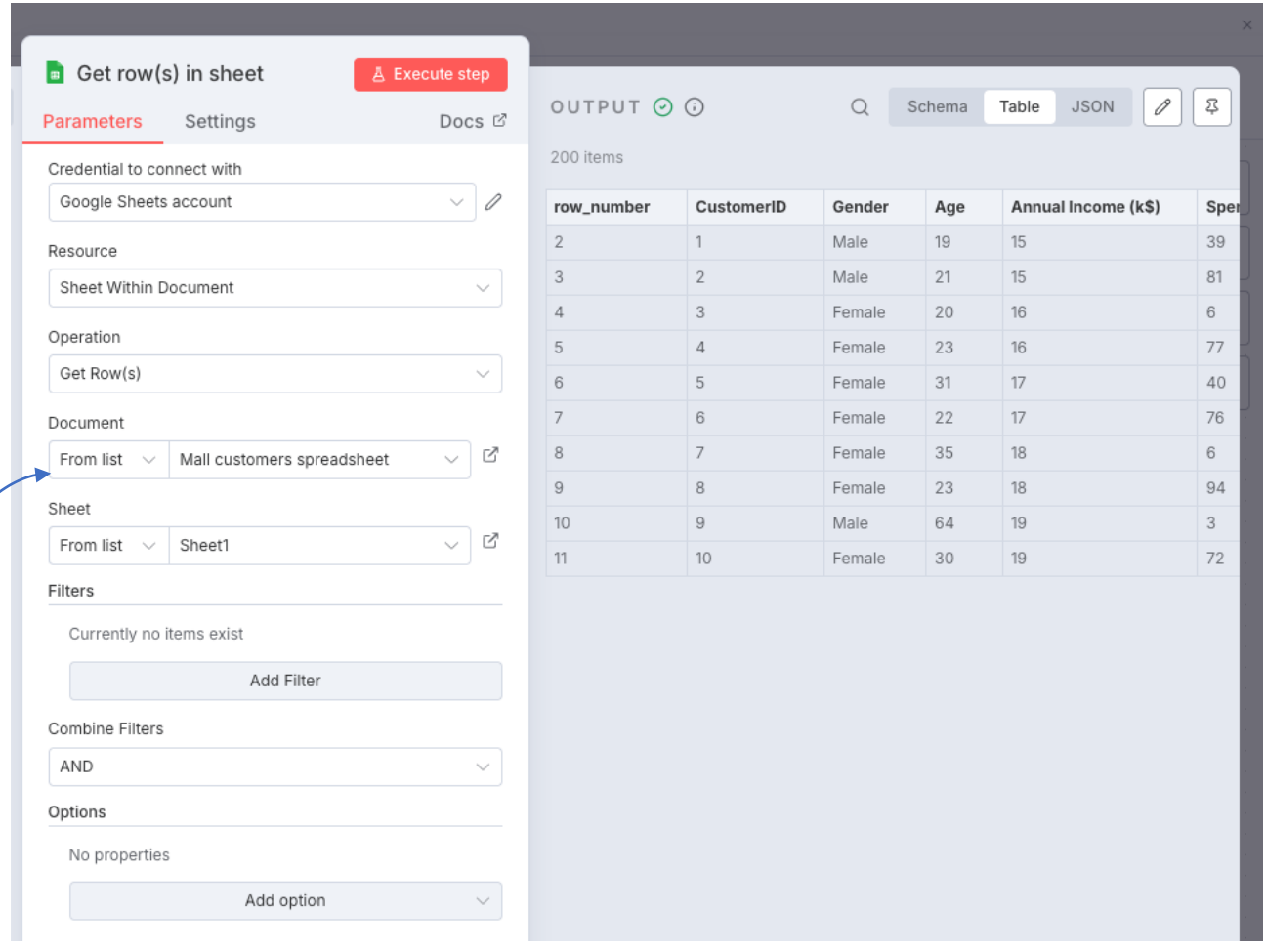
¡VAMOS CON EL SIGUIENTE NODO, QUE SERÁ DE GOOGLE SHEETS!

- Estableceremos una credencial vinculada a nuestra cuenta de Google, que podrá buscar, en el sistema de archivos **Google Drive** de dicha cuenta, ficheros de hojas de cálculo
- Elegiremos dentro de 'Google Sheets', la acción "get row(s) in sheet", que lee datos de una hoja de cálculo

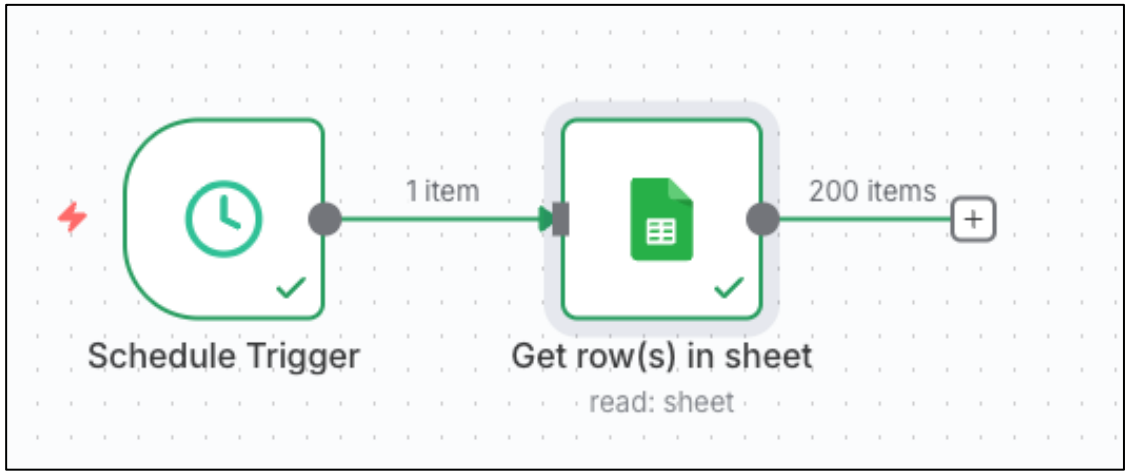
Aquí selecciona tu recién creada credencial

Aquí, la primera vez es posible que te aparezca un error, y deberás acceder a la URL que te indique en Google Cloud para habilitar la API de Google Drive que permitirá buscar hojas de cálculo en tu Drive





Si todo ha ido bien, tendremos ya 2 nodos, y el segundo nos devuelve un conjunto de filas extraídas de la hoja de cálculo



- En la demo de clase, habremos recuperado 200 registros de clientes del dataset *Mall Customers*
- Vamos a añadir también un nodo de código, donde se puede añadir lógica y funciones en Javascript o código (beta)



Código para el nodo de Javascript que analizará los datos recuperados de Google Sheets y obtendrá frecuencias de clientes por intervalos de edad (1/2)

```
// Obtener todos los items del flujo entrante en el nodo
const items = $input.all();

// --- PASO 1: Extracción del atributo 'Age' (edades de clientes) --- //

// Función que devuelve el primer campo numérico que corresponda con la edad
function extractAge(obj) {
  if (!obj || typeof obj !== 'object') return null;

  // Se busca de modo que no haya sensibilidad a mayúsculas y minúsculas
  for (const key of Object.keys(obj)) {
    if (key.toLowerCase() === "age") {
      const val = Number(obj[key]);
      return !isNaN(val) ? val : null;
    }
  }
```

```
 // Probar por si acaso el objeto JSON está anidado
  if (obj.json && typeof obj.json === "object") {
    return extractAge(obj.json);
  }

  return null;
}

const ages = items
  .map(item => extractAge(item.json ?? item))
  .filter(a => typeof a === "number" && !isNaN(a));

// Si no hay datos recuperados, crear mensaje de error para siguiente nodo de QuickChart
if (ages.length === 0) {
  return [
    {
      json: {
        labels: ["No data"],
        data: [0],
        debug_note: "No ages found. Check column name (expected: 'age' or 'Age')."
      }
    }
  ];
}
```

Código para el nodo de Javascript que analizará los datos recuperados de Google Sheets y obtendrá frecuencias de clientes por intervalos de edad (2/2)

```
// --- PASO 2: Construcción dinámica de 10 intervalos de edad (bins) --- //

const BIN_SIZE = 10;

const minAge = Math.min(...ages);
const maxAge = Math.max(...ages);

const start = Math.floor(minAge / BIN_SIZE) * BIN_SIZE;
const end = Math.ceil(maxAge / BIN_SIZE) * BIN_SIZE;

let labels = [];
let data = [];

for (let binStart = start; binStart < end; binStart += BIN_SIZE) {
  const binEnd = binStart + BIN_SIZE - 1;

  labels.push(`${binStart}-${binEnd}`);

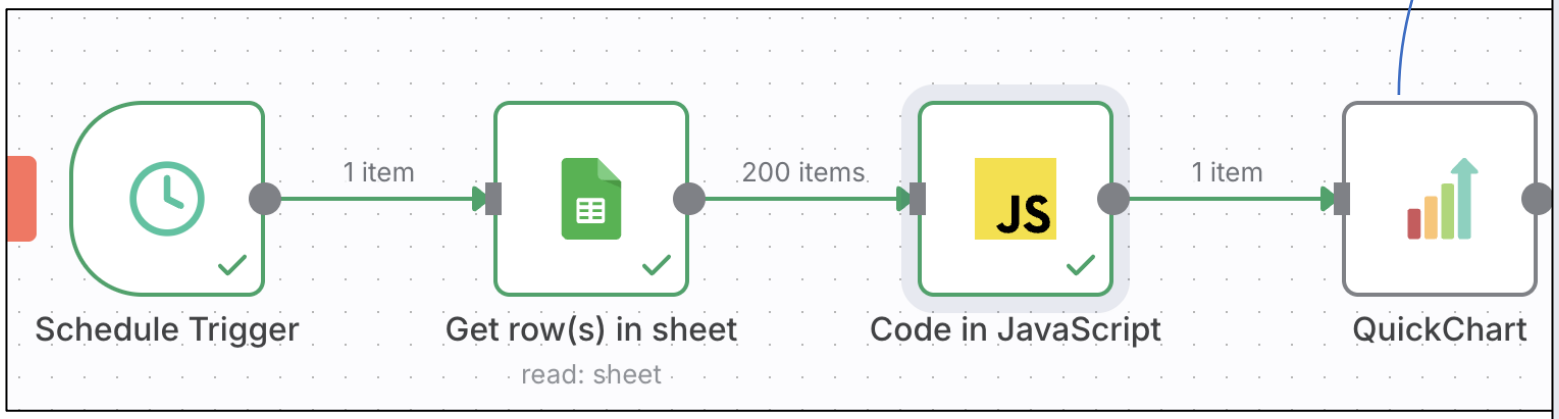
  const count = ages.filter(a => a >= binStart && a <= binEnd).length;
  data.push(count);
}
```

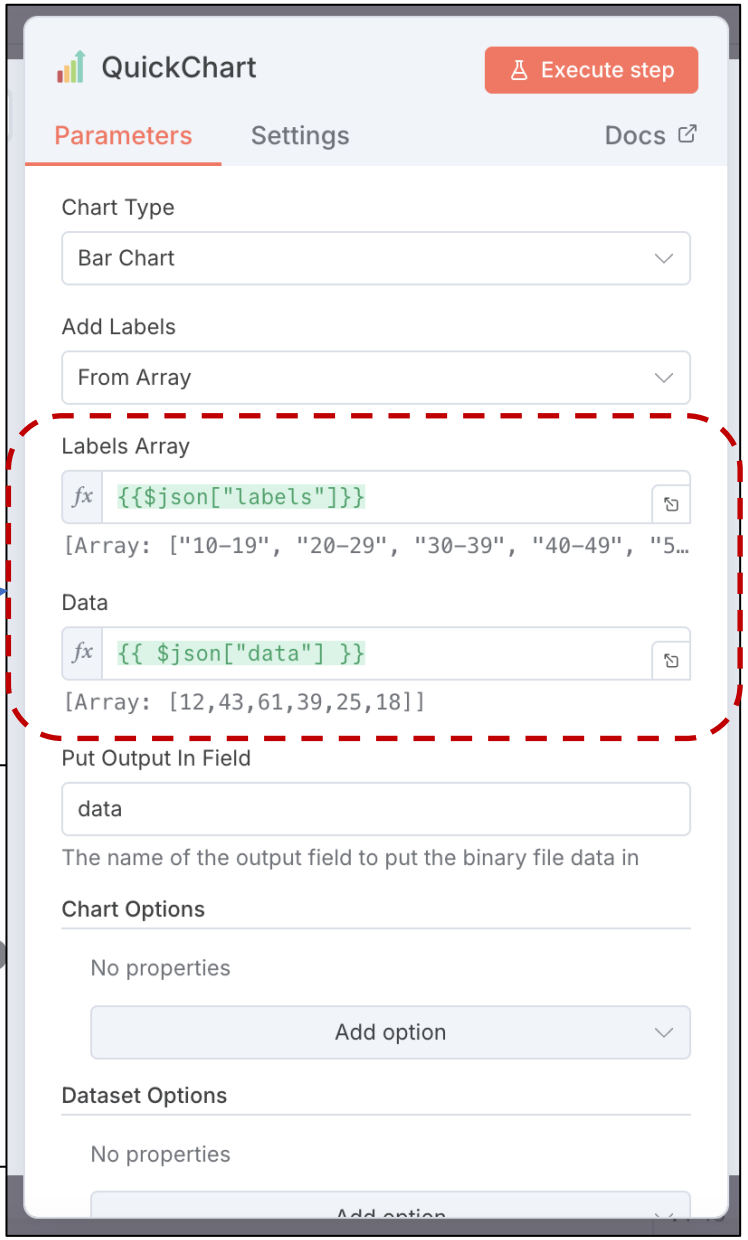
```
// --- PASO 3: Devolver objeto JSON apto para posterior nodo de QuickChart --- //

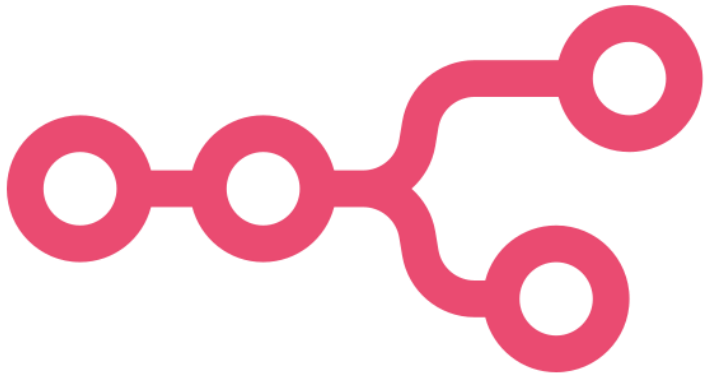
return [
  {
    json: {
      labels,
      data
    }
  }
];
```

Para terminar, crearemos un nodo QuickChart, que sirve para crear un gráfico de visualización “al vuelo” con los datos entrantes al nodo

Asegúrate de activar el modo “expresión” (f_x)
Para editar estos dos campos







n8n

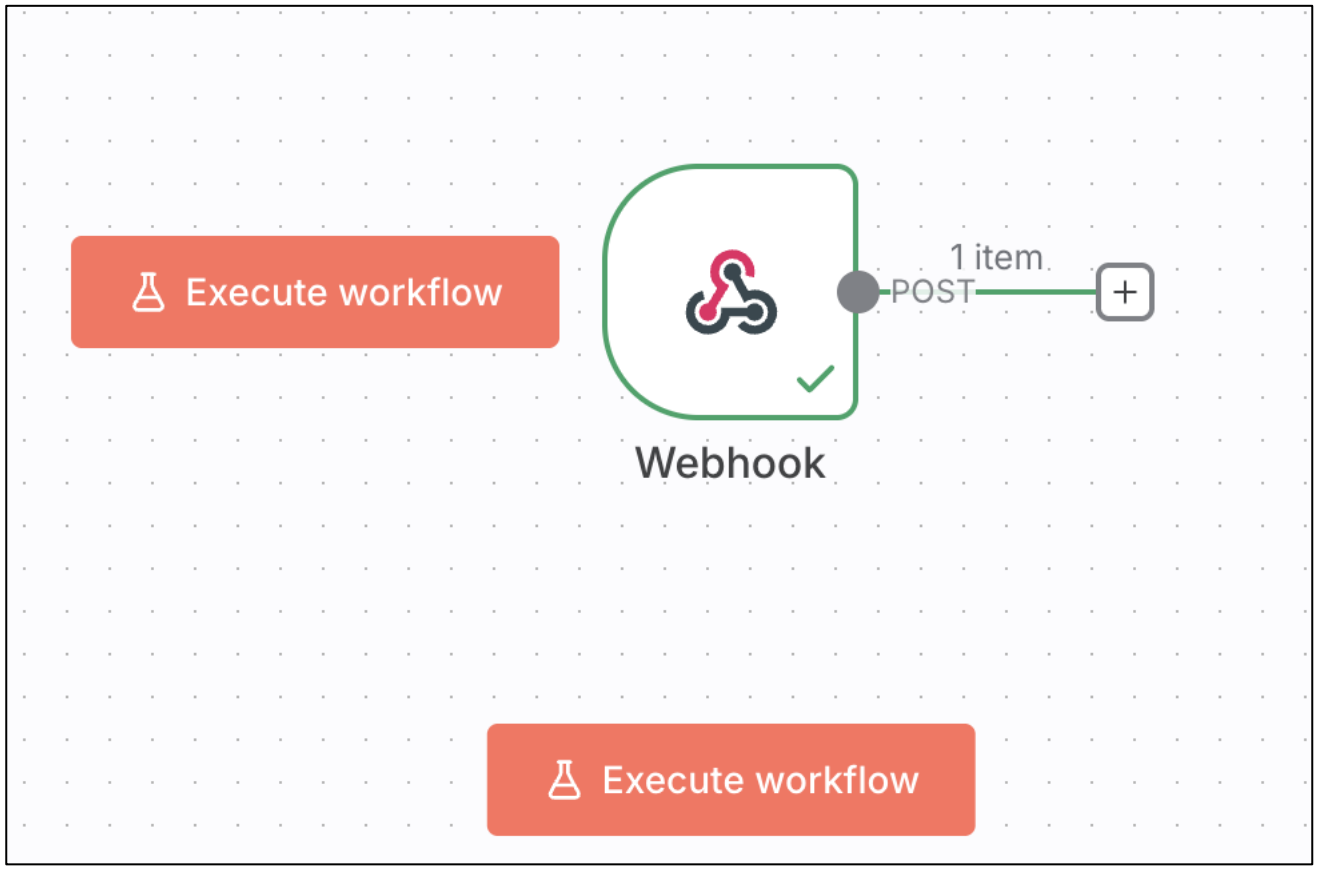


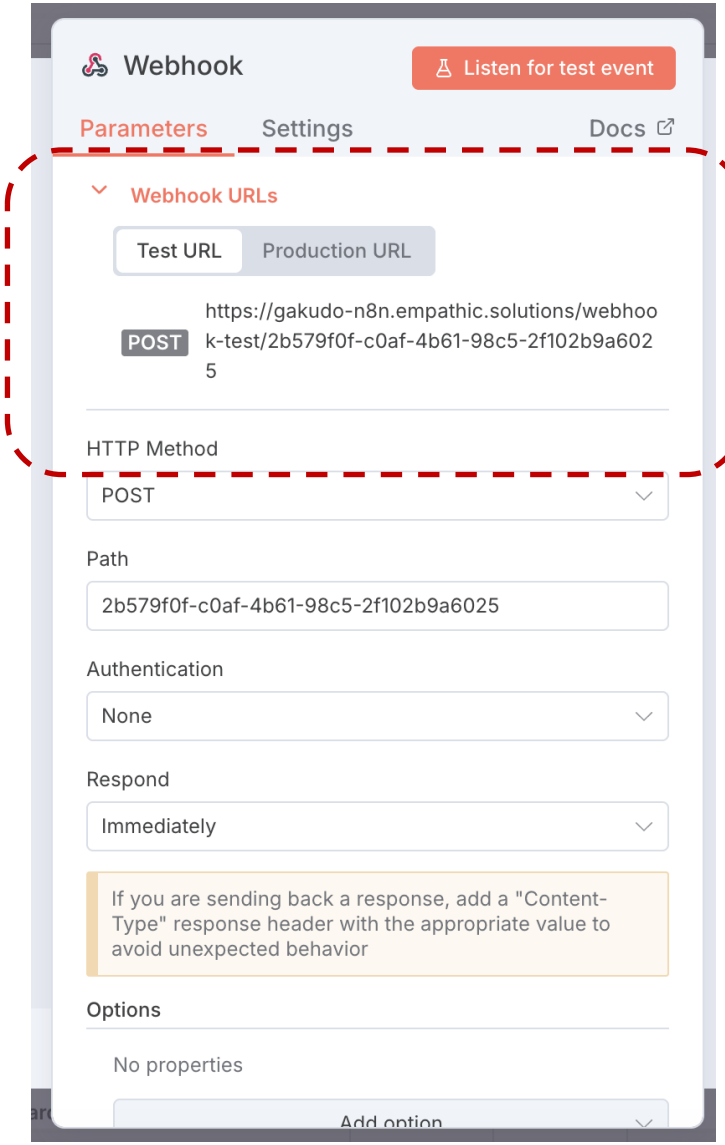


OpenAI

Vamos a crear un flujo que integra acceso a un modelo de ChatGPT mediante la API de OpenAI

- El flujo comenzará con un **nodo *trigger* del tipo Webhook**, que debe ser configurado para recibir peticiones web de tipo POST desde el exterior.

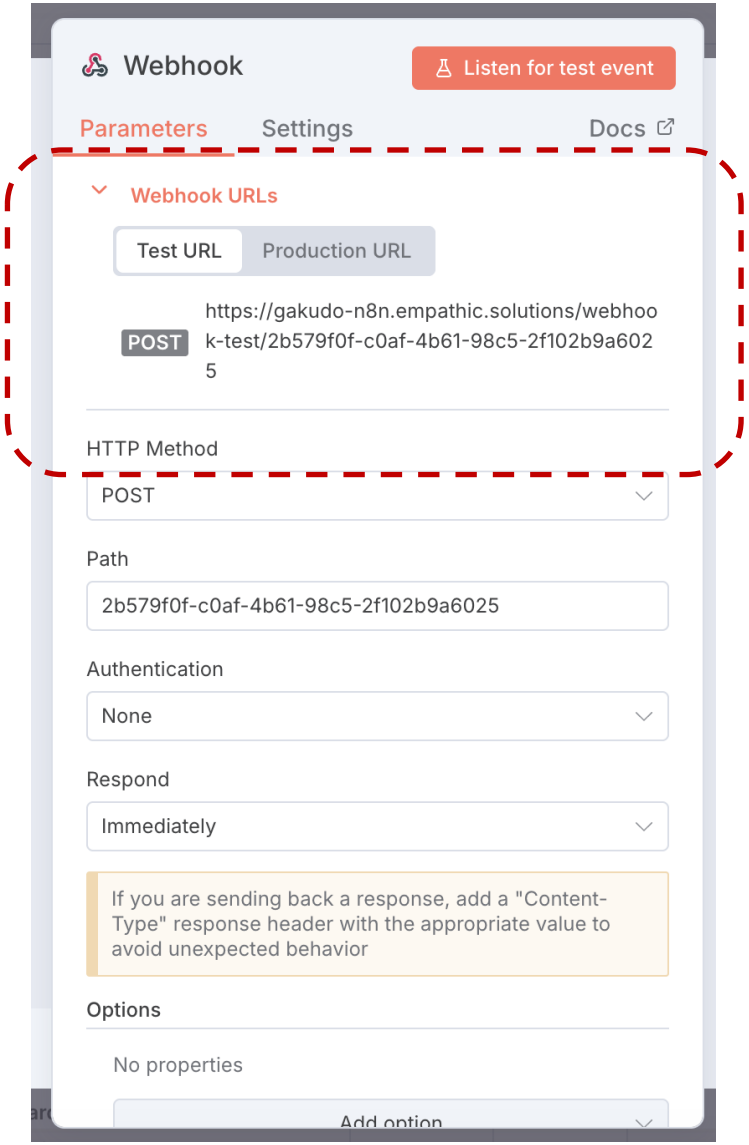




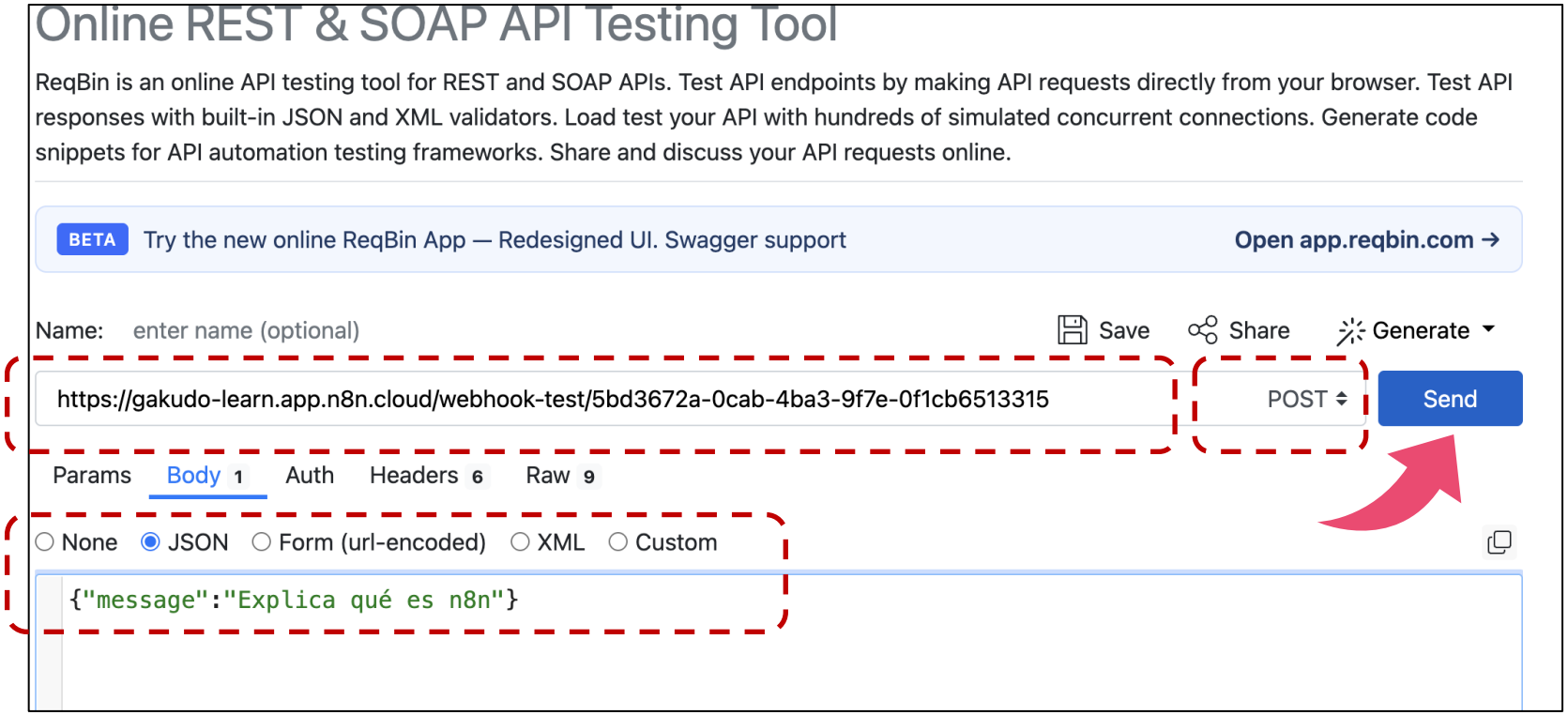
Un posible modo de enviarle una petición POST externa al nodo trigger de nuestro flujo en n8n, es através de un comando *curl* en nuestra propia terminal

OJO: en un entorno de producción donde Quieras enviar peticiones sucesivas POST, deberás cambiar "webhook-test" por "webhook" (Production URL)

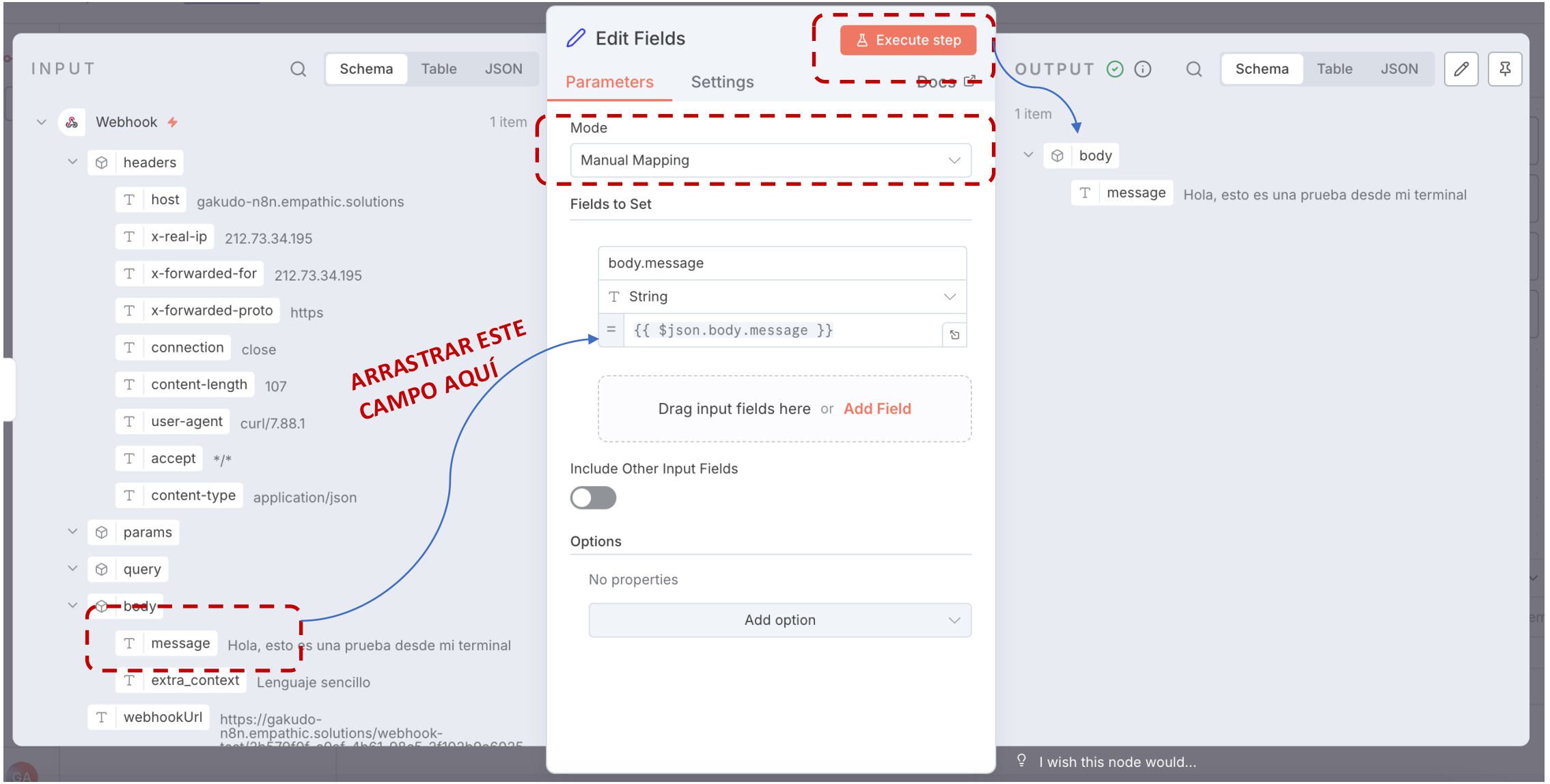
```
curl -X POST https://gakudo.app.n8n.cloud/webhook-test/5bd3672a-0cab-4ba3-9f7e-0f1cb6513315 \
-H "Content-Type: application/json" \
-d '{"test": "success", "message": "Explica los conceptos básicos de n8n para automatización de procesos sencillos."}'
```



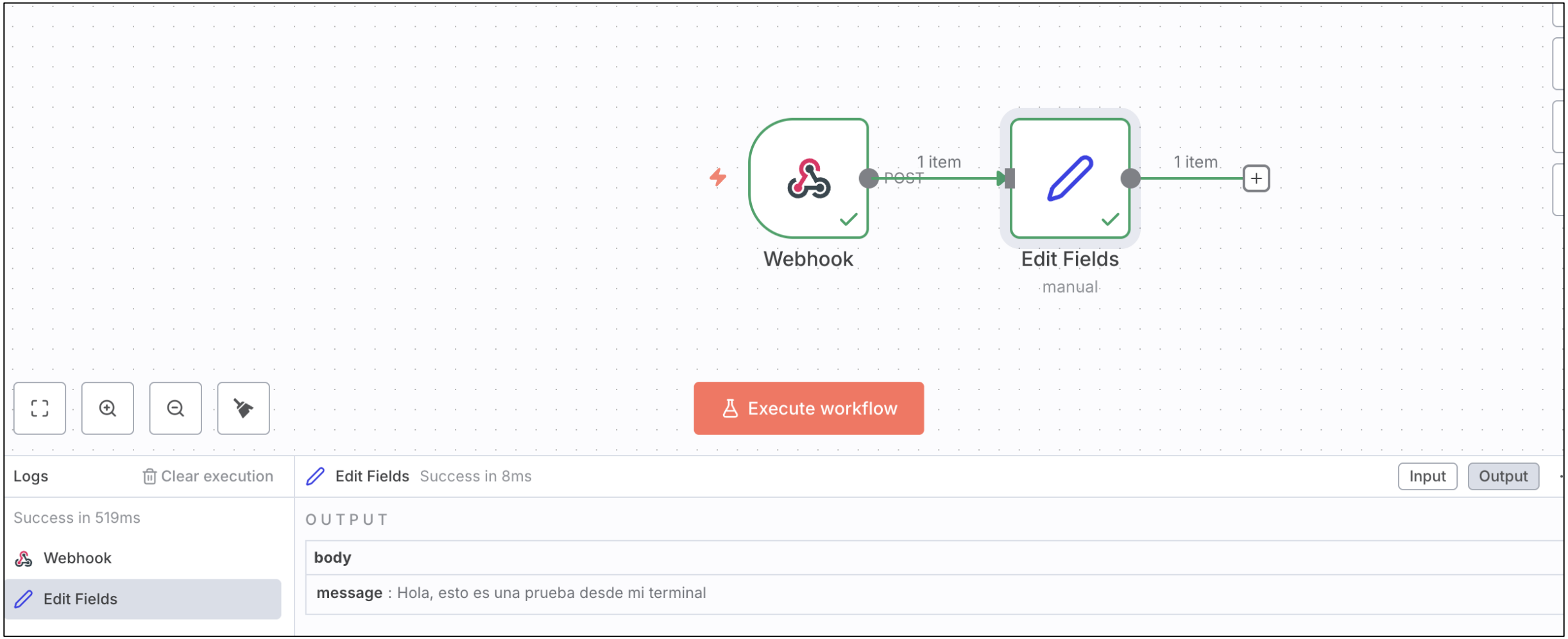
Otro modo más sencillo es desde REQBIN, una Web que permite testar servicios Web, incluyendo peticiones GET, POST, etc.: https://reqbin.com/
Sin necesidad de escribir comandos curl desde tu terminal.



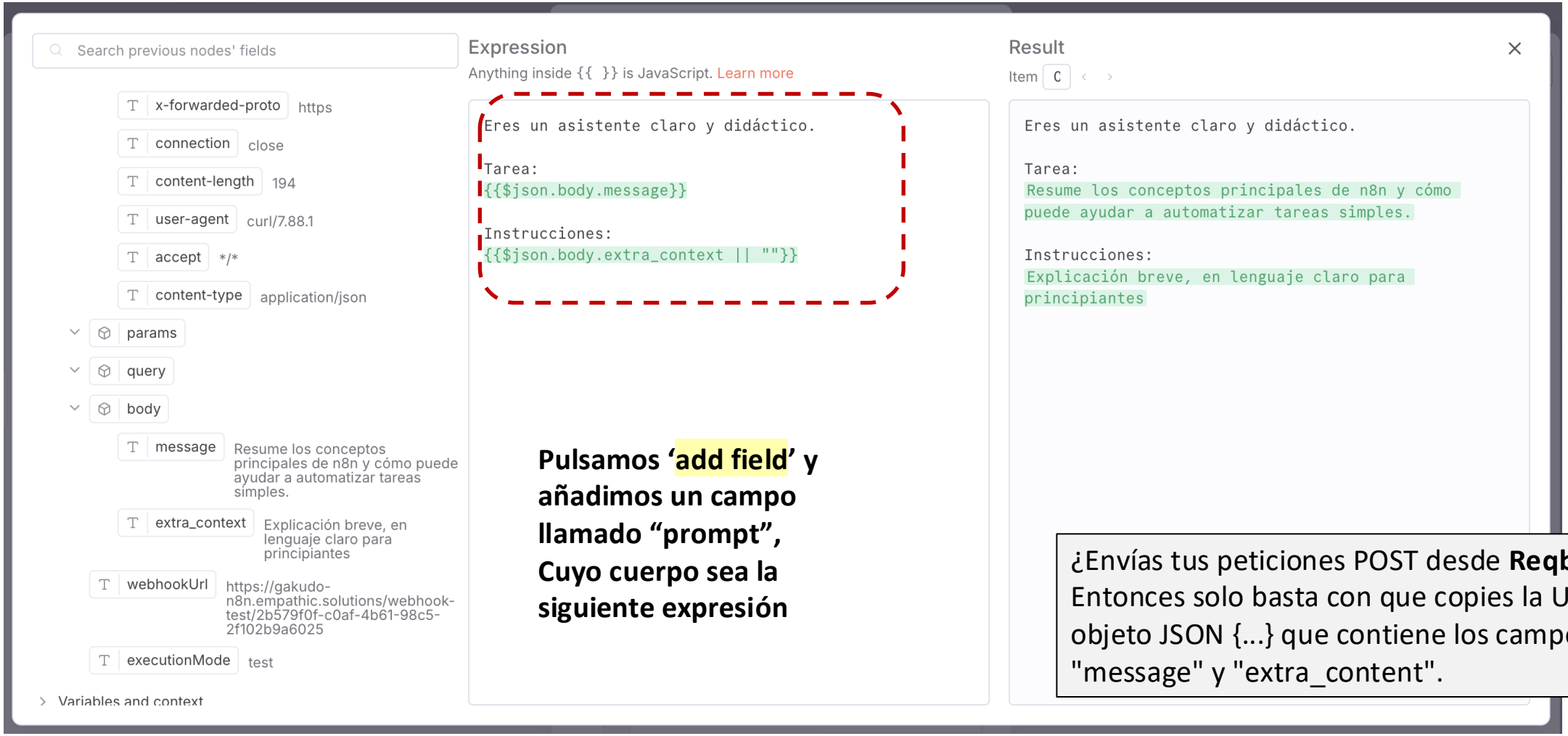
Siguiente nodo, del tipo "Edit Fields (Set)": extrae lo que será nuestro prompt para ChatGPT posteriormente



En este punto deberíamos tener algo como esto (atención a la salida obtenida tras el segundo nodo):



Vamos a configurar en nuestro segundo nodo, del tipo **Edit Fields**, para preparar un buen prompt que enviaremos al LLM de OpenAI (un modelo de ChatGPT). Empecemos por actualizar la petición POST:

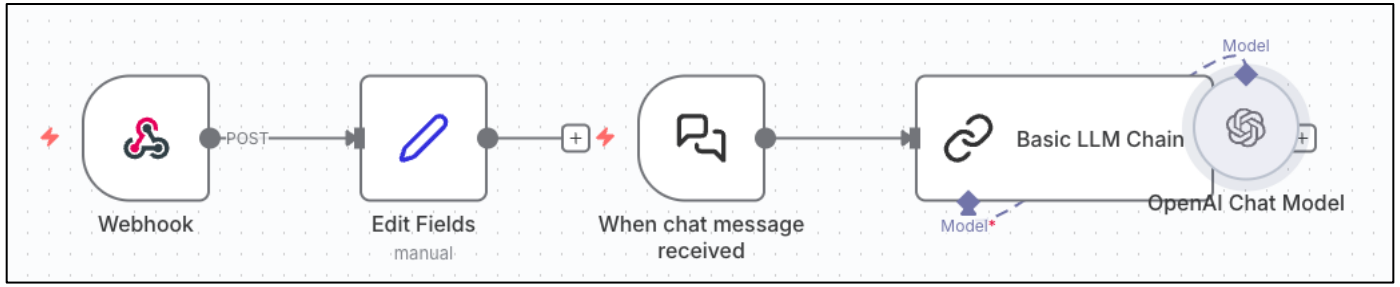


Pulsamos 'add field' y añadimos un campo llamado "prompt", Cuyo cuerpo sea la siguiente expresión

¿Envías tus peticiones POST desde **Reqbin**? Entonces solo basta con que copies la URL y el objeto JSON {...} que contiene los campos "message" y "extra_content".

Conjunto de nodos "Open AI Chat Model"

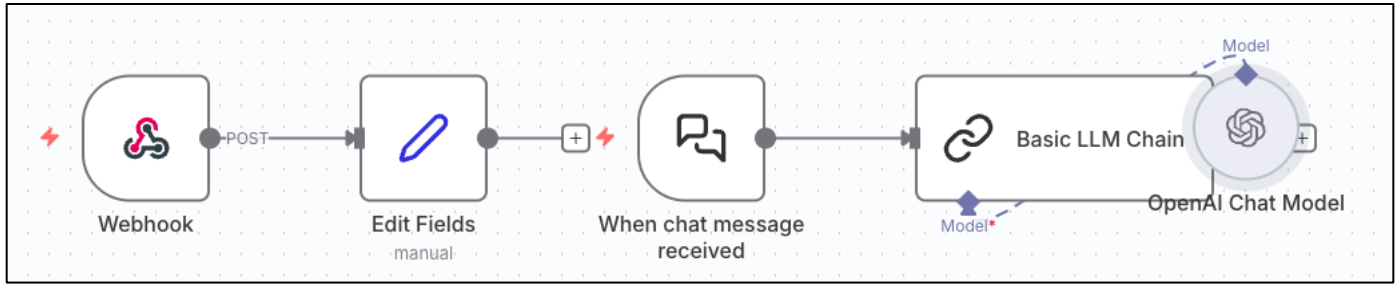
- Cuando creamos un nodo de este tipo, en realidad se nos despliega en el flujo un grupo de 3 nodos.



- El primero, "***when chat message received***", pese a no ser el primer nodo del workflow, es un *trigger*, que espera un evento para reaccionar
- → No lo necesitaremos aquí: lo eliminamos y estiramos la arista hacia el siguiente nodo: "***Basic LLM Chain***"

Conjunto de nodos "Open AI Chat Model"

- Cuando creamos un nodo de este tipo, en realidad se nos despliega en el flujo un grupo de 3 nodos.

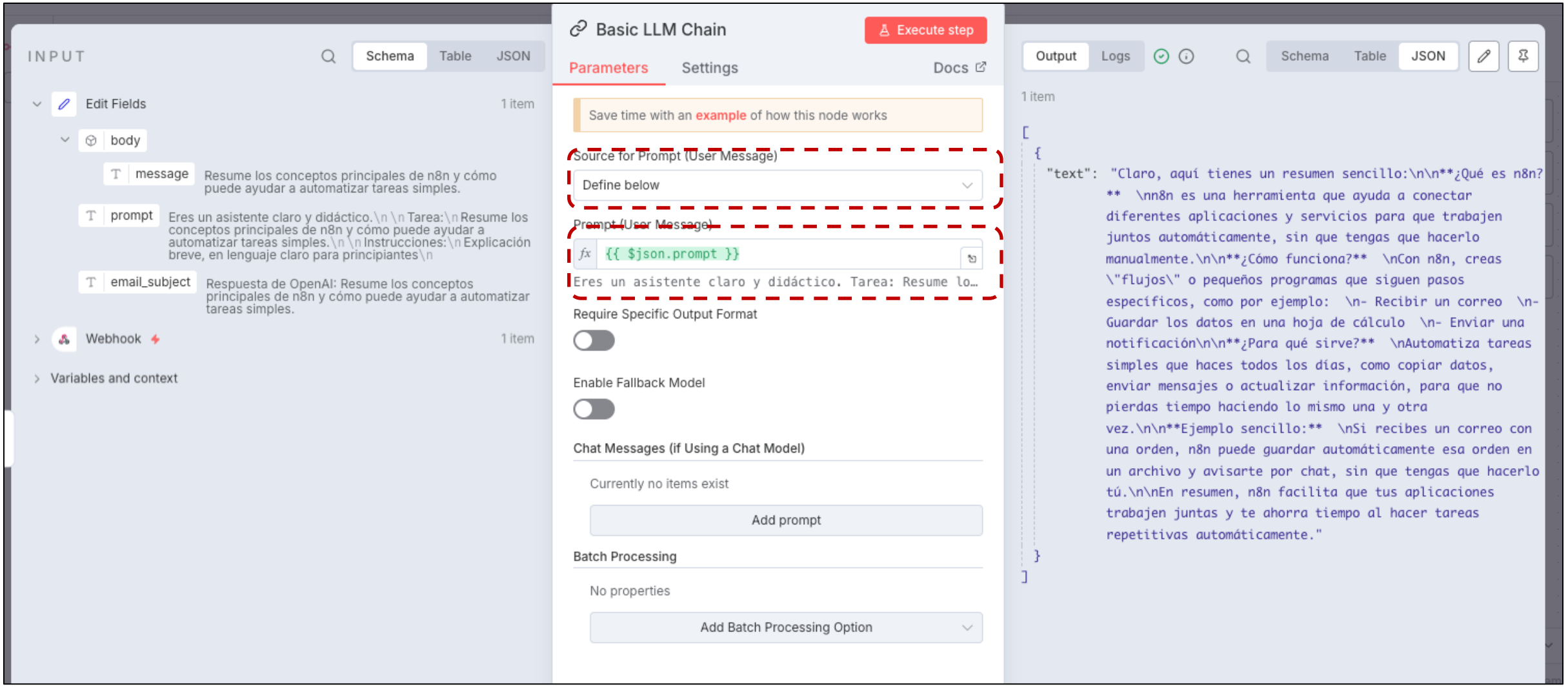


- El primero, "***when chat message received***", pese a no ser el primer nodo del workflow, es un *trigger*, que espera un evento para reaccionar
- → No lo necesitaremos aquí: lo eliminamos y estiramos la arista hacia el siguiente nodo: "***Basic LLM Chain***"



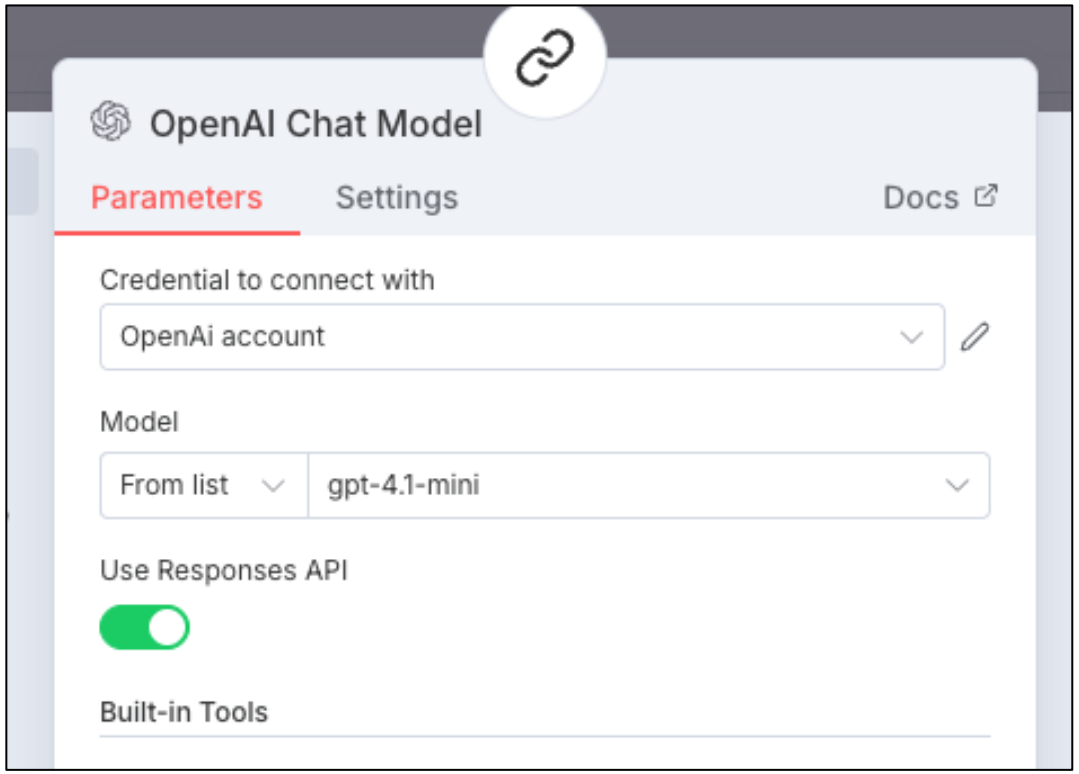
El nodo BASIC LLM Chain

- Se encarga de **orquestar la interacción con el LLM** externo de OpenAI, **construyendo** a partir de datos entrantes el que será el **prompt final para el modelo.**



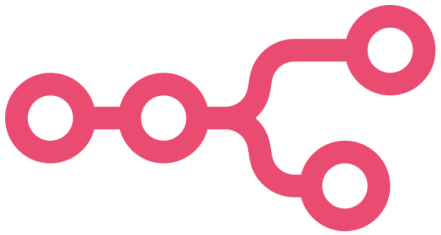
El nodo OpenAI Chat Model

- Va adjunto al 'Basic LLM Chain': mientras que aquel era el "cerebro" de la operación, este es el "brazo ejecutor" → seleccionar un modelo. **¡CUIDADO: según el momento, algunos modelos no se podrán usar con cuenta OpenAI gratuita!**



Deberás añadir en n8n una credencial para acceso a OpenAI. Lo más importante a obtener es una API Key de OpenAI, accesible en: https://platform.openai.com/settings/organization/billing/overview

¡OJO AL DATO!
N8n cloud actualmente ofrece **créditos gratuitos (Gateway Credits) para usar modelos de OpenAI y Google Gemini**, sin necesidad de definir nuestra propia credencial : ¿por qué no aprovechar esta oportunidad en clase?

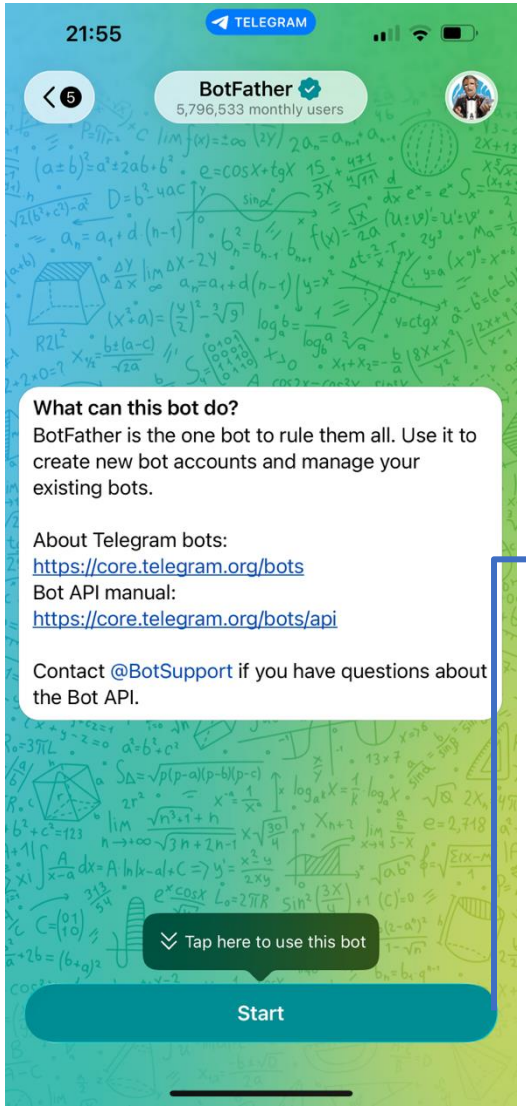


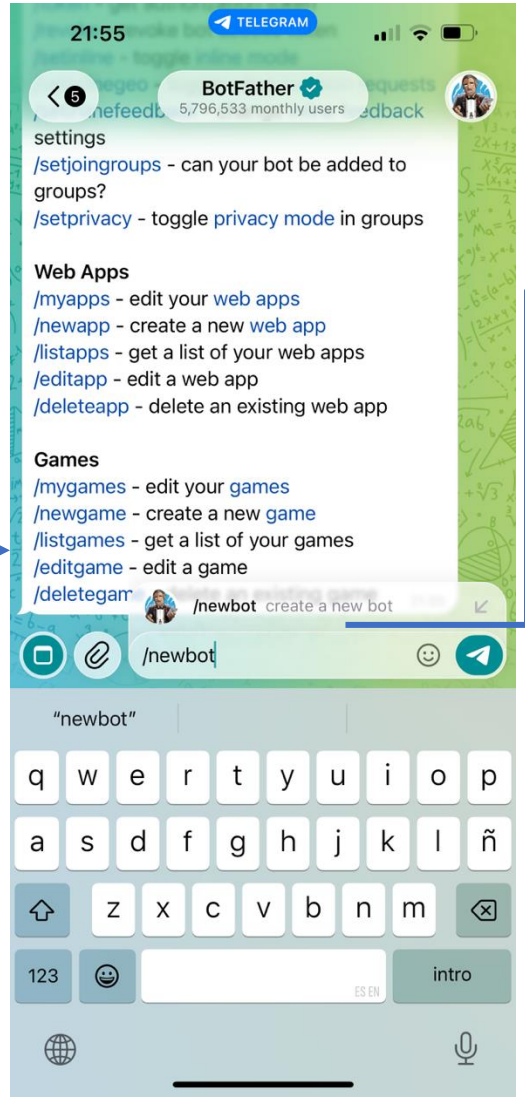
n8n

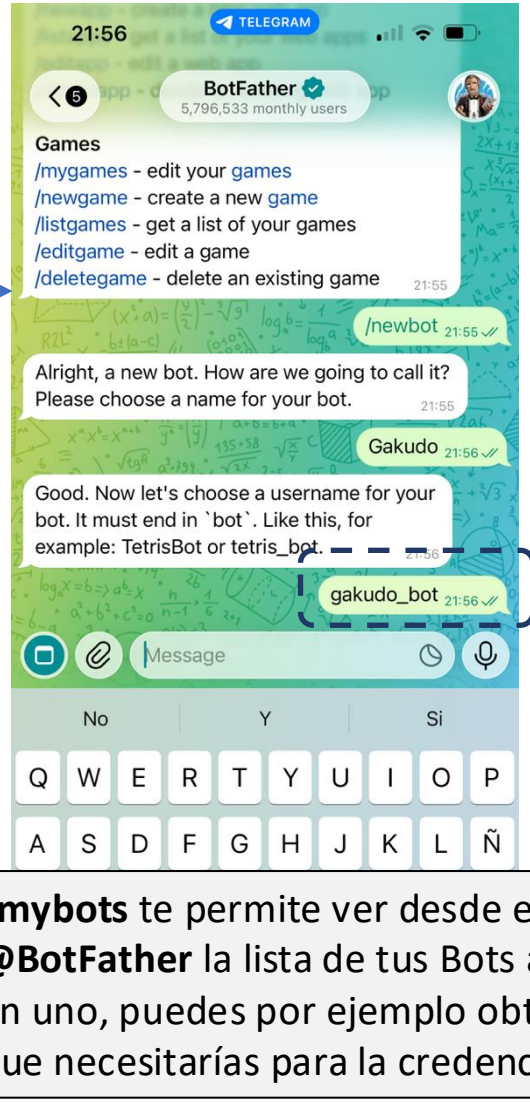




En TELEGRAM, buscaremos y añadiermos la cuenta @BotFather (el Bot oficial de Telegram)



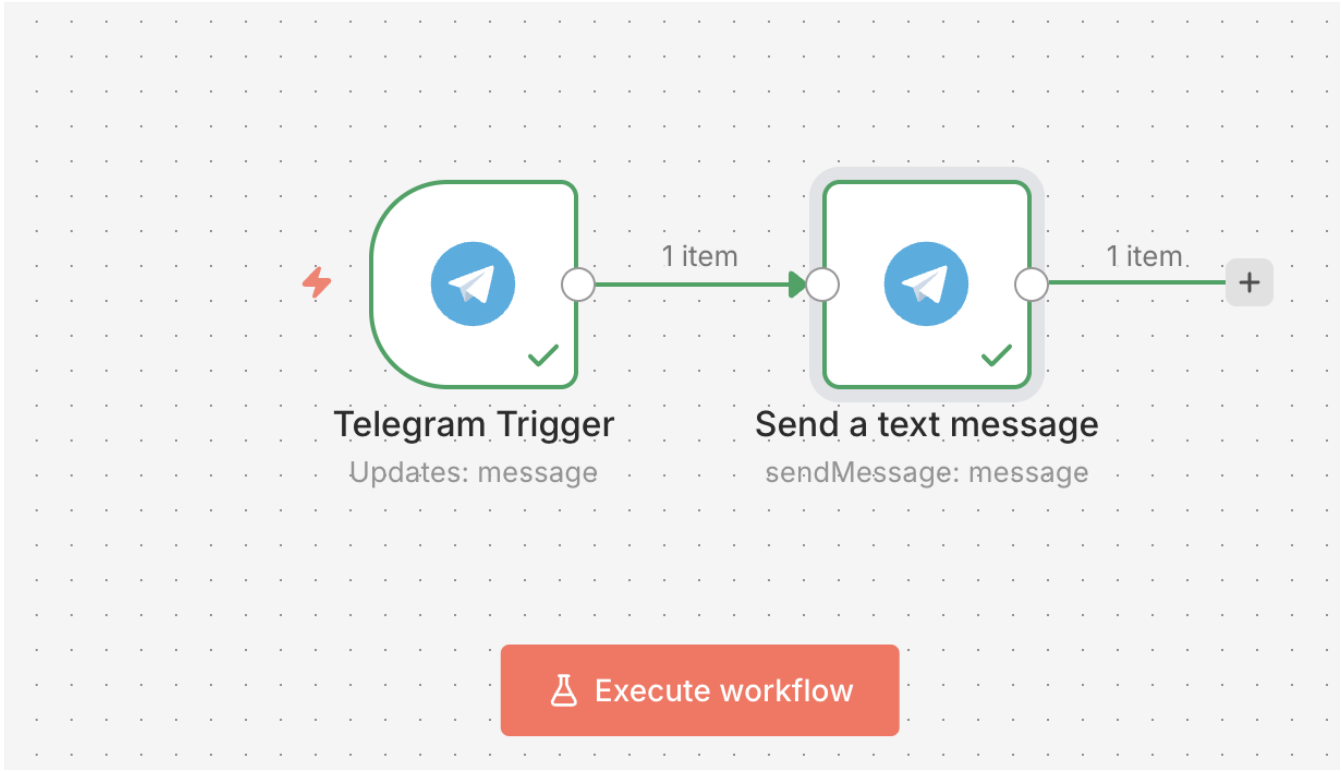


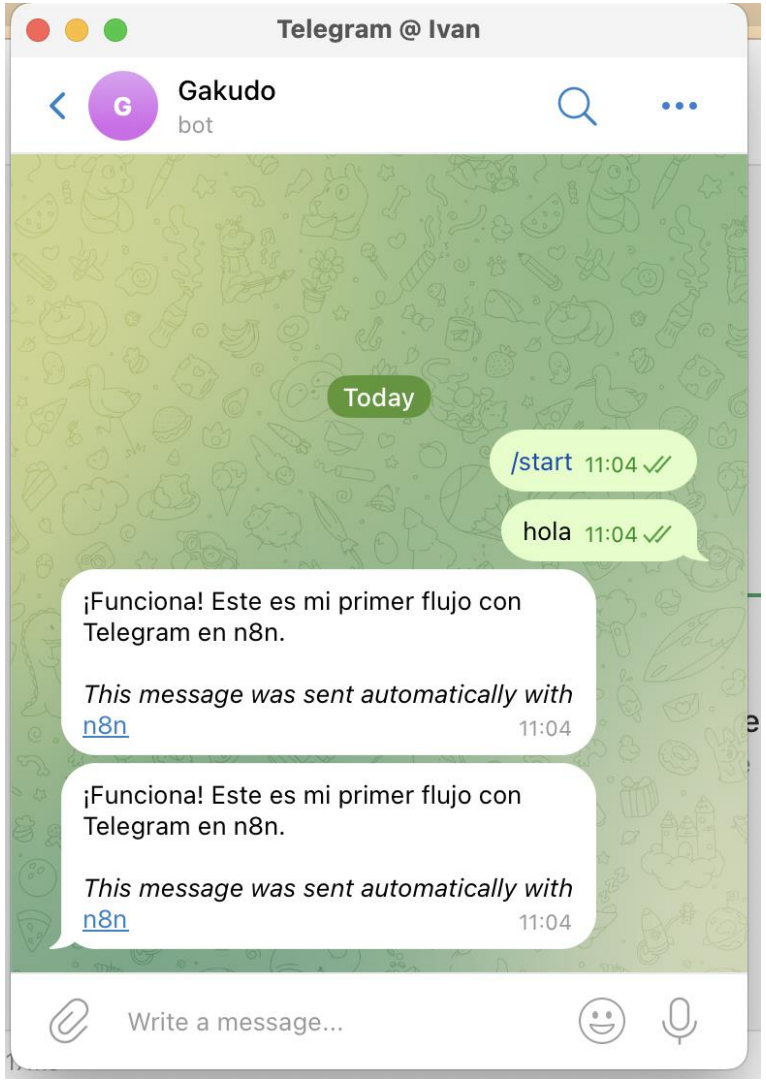


IMPORTANTE: este *username* Que elijas se usará para crear un nuevo usuario 'bot' en Telegram que debes buscar y empezar a charlar con él después

/mybots te permite ver desde el chat con **@BotFather** la lista de tus Bots activos. Haciendo clic en uno, puedes por ejemplo obtener el API Token que necesitarías para la credencial en n8n

Hablaremos con nuestro recién creado bot, para recibir de inmediato una respuesta automática generada desde nuestro recién creado flujo en n8n





El "misterio" del formato JSON (y por qué en realidad "no muerde")

JSON ≈ Paquete que contiene datos

- Se usa en n8n porque es el estándar que las aplicaciones en Internet utilizan para enviarse información
- Todo objeto en JSON contiene elementos con una estructura **"etiqueta· : "valor"** (Key-Value).
- **Ejemplo:** "nombre": "María" → la etiqueta o nombre de la variable es "nombre", el dato en sí, o valor de esa variable, es un *String*: "María".

1. Objeto Simple (La caja individual)	**2. Objeto con Array(s) dentro**	**3. El Array de Objetos (el más comúnmente empleado en n8n)**
Representa a **una sola entidad** con datos directos. Es el formato más básico.	Representa a una entidad que tiene **una lista de elementos** en su interior ([ ]).	Es una **lista que contiene múltiples objetos JSON** separados. Así es como n8n mueve los datos entre nodos.
json { "nombre": "María", "edad": 35, "suscrito": true }	**json** { "cliente": "Empresa XYZ", "etiquetas": ["VIP", "Formación", "2026"] }	**json [** { "id": 1, "nombre": "María" }, { "id": 2, "nombre": "Juan" } **]**

El "misterio" del formato JSON (y por qué en realidad "no muerde")



Anatomía del dato en n8n



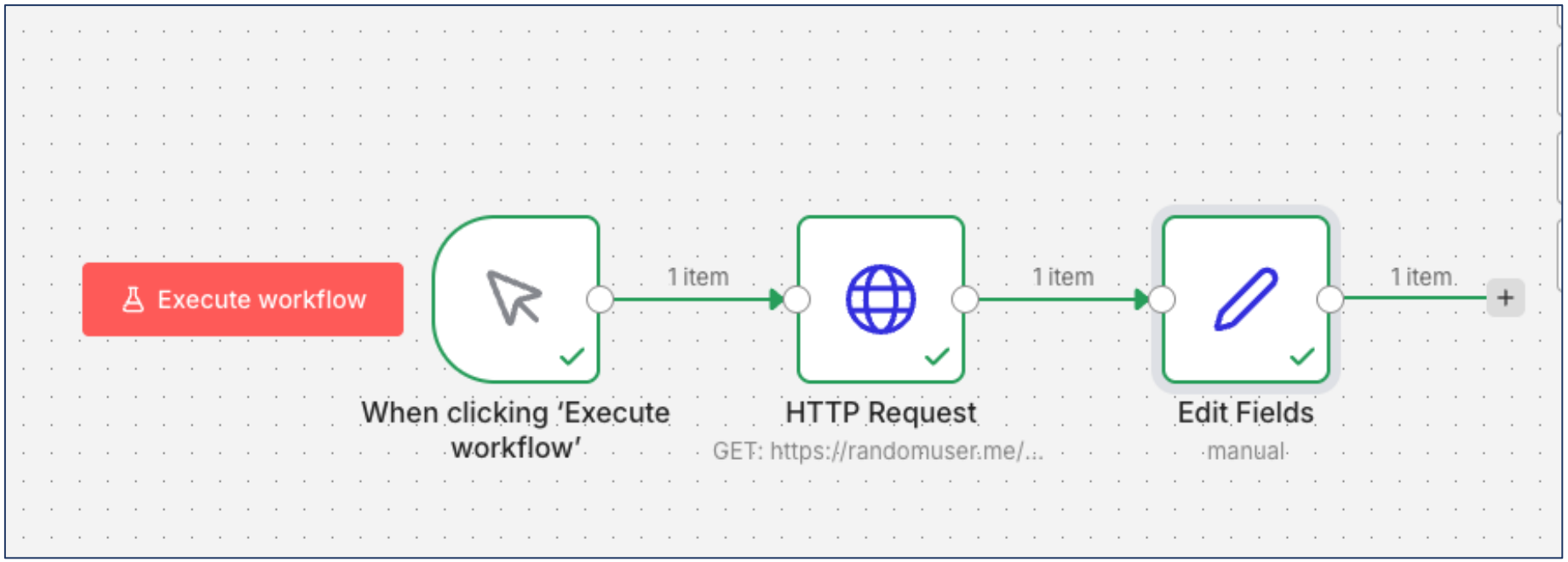
Las listas [, ,] y objetos o ítems { ... } son como un tren de mercancías y sus vagones.

Elementos clave:

- **Objetos { }** : delimitados por llaves. Representan un elemento, que dentro de sí puede tener uno o varios atributos, y cada atributo puede ser un tipo de dato simple (número, texto, etc.), otro objeto anidado, o una lista/colección de elementos.
- **Listas [ ]:** delimitadas por corchetes. Representan una colección de elementos, p.ej. una lista de 200 clientes, siendo cada cliente un objeto que tendrá anidados varios atributos.

En n8n, la información SIEMPRE viaja por los flujos como una lista [ ] de objetos { } incluso si es de un solo elemento.

EJEMPLO GUIADO: MAPEO Y MANIPULACIÓN DE DATOS JSON



PARA INVESTIGAR MÁS SOBRE MANIPULACIÓN DE DATOS EN JSON MEDIANTE EXPRESIONES:

https://docs.n8n.io/code/expressions/

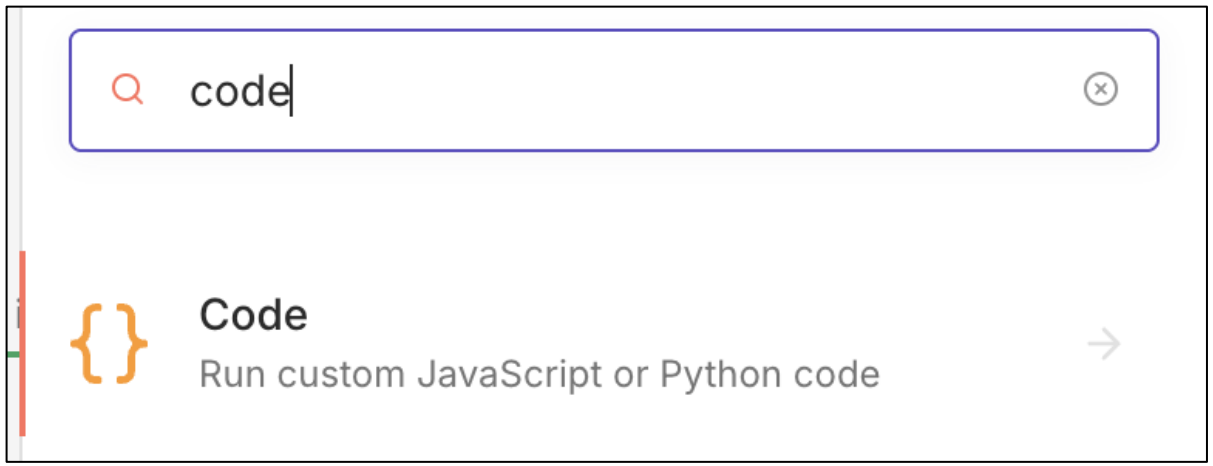
https://docs.n8n.io/code/builtin/

https://developer.mozilla.org/es/docs/Web/JavaScript/Reference/Operators/Conditional_operator

https://developer.mozilla.org/es/docs/Web/JavaScript/Reference/Global_Objects/Math

En n8n, el uso de código Python está en parte limitado, ya que solo se permite el uso de librerías estándar o nativas del lenguaje.

- Librerías altamente populares de terceros, como la de **Pandas** para la manipulación de datos, quedan fuera del alcance de los nodos de código en n8n.
- Pero con un poco de creatividad, la **librería estándar de Python** puede llegar a ser una pequeña "navaja suiza" para manejar datos en situaciones en las que Javascript podría resultar más tedioso.



FUNCIONES PYTHON EN n8n

- **Idea clave:** para que el código Python funcione, normalmente toda función deberá devolver una lista de diccionarios Python.

- De este modo, un "Hola Mundo" (programa muy simple de iniciación nuevo lenguaje de programación o entorno), sería algo así como:

```
# Creamos un paquete JSON nuevo para el siguiente nodo
return {
    "mensaje": "¡Hola, acabo de programar en Python!"
}
```

FUNCIONES PYTHON EN n8n

- **Idea clave:** para que el código Python funcione, normalmente toda función deberá devolver una lista de diccionarios Python.

- De este modo, un "Hola Mundo" (programa muy simple de iniciación nuevo lenguaje de programación o entorno), sería algo así como:

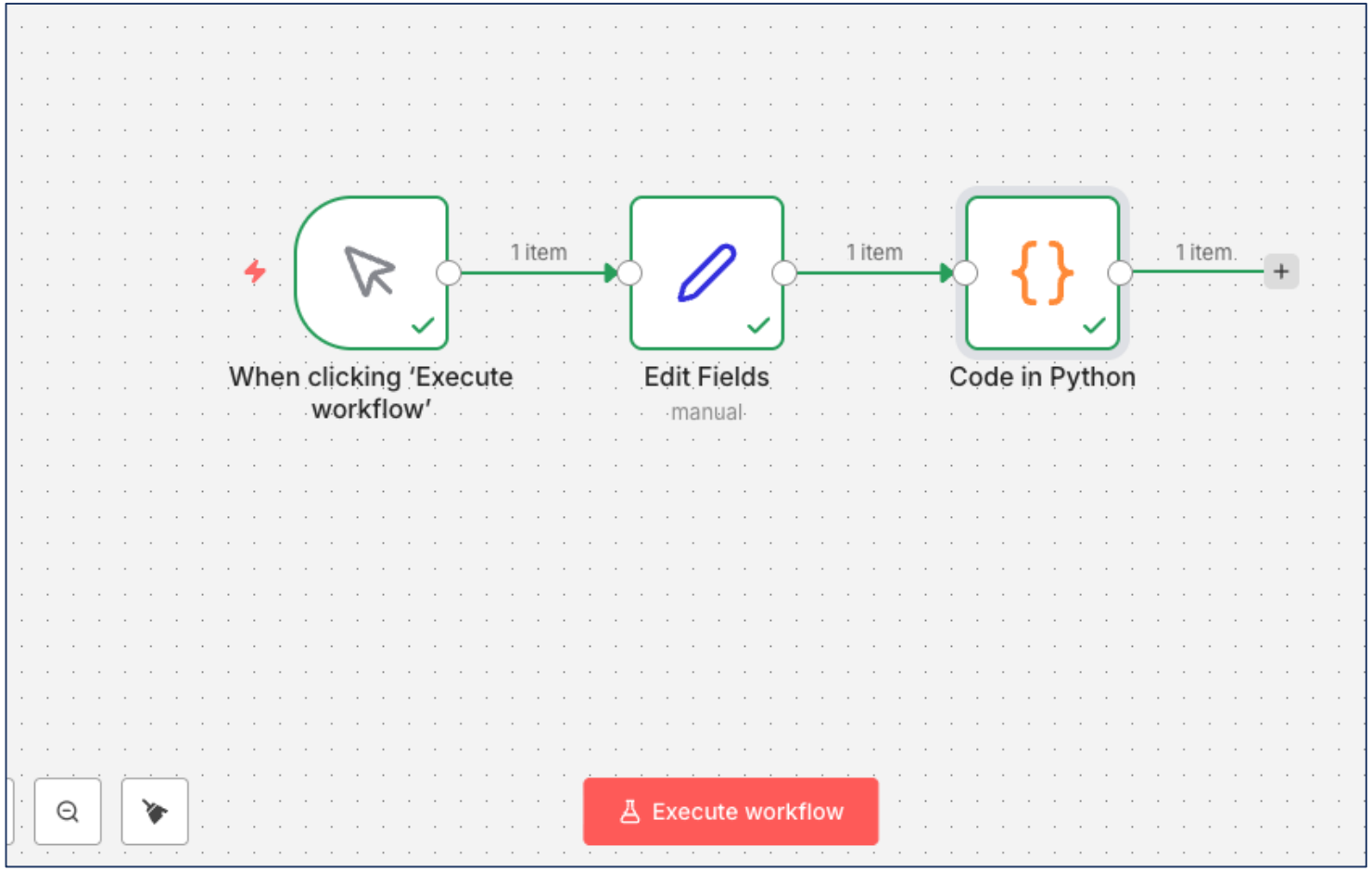
```
# Código compatible con n8n Native Python
for item in _items:
    # Usamos .get() con corchetes para mayor seguridad
    nombre = item["json"].get("cliente", "desconocido")

    # Aplicamos la limpieza y formato
    limpio = nombre.strip().title()

    # Guardamos el resultado usando acceso por
corchetes
    item["json"]["saludo_personalizado"] = f"¡Hola
{limpio}! Bienvenido al curso."

return _items
```

¡APRENDEREMOS UN POCO MÁS DEL LENGUAJE PYTHON A TRAVÉS DE ALGUNOS EJEMPLOS!



FUNCIONES JAVASCRIPT EN n8n

- **Más flexible que Python** dentro del ecosistema de n8n, pero resulta un lenguaje más complejo de utilizar e interpretar.
- El motivo es que n8n está construido sobre una tecnología basada en Javascript: **Node.js**
- Además, como bien sabemos ya, n8n maneja los datos en formato JSON por defecto.

Javascript suele requerir mas líneas de código para hacer una misma cosa:

Extraer el alias de usuario de una dirección de email.

ARRIBA: Python
ABAJO: Javascript

```python
# Modo: Run once for each item
# Procesa el email y crea el campo 'nombre'
item['nombre'] = item['email'].split('@')[0].upper()
return item
```

```javascript
// Modo: Run once for each item
// Requiere pasos intermedios para la misma lógica
const email = $input.item.json.email;
const nombreExtraido = email.split('@')[0];
$input.item.json.nombre = nombreExtraido.toUpperCase();

return $input.item;
```